

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
(ШКОЛА)

Департамент программной инженерии и искусственного интеллекта

О Т Ч Е Т
о прохождении учебной практики
технологической (проектно-технологической)

Выполнил студент гр. Б9125-09.03.04пи

(подпись) А.А. Расторгуев
(ФИО студента)

Отчет защищен с оценкой

Руководитель практики
доцент департамента ПИИИИ, к.ф.-м.н.
(должность, уч. степень, уч. звание)

(подпись) Ю.Е. Иванова
(И.О. Фамилия руководителя)
« 04 » _____ июля 2026 г.

(подпись) Ю.Е. Иванова
(И.О. Фамилия руководителя)

Регистрационный №

Практика пройдена в срок
с « 19 » _____ февраля 2026 г.
по « 04 » _____ июля 2026 г.
на предприятии

« _____ » _____ 2026 г.

_____ ДВФУ

(подпись) _____
(И.О. Фамилия)

Владивосток
2026

Оглавление

1 Введение.....	3
2 Основная часть	4
2.1 Описание проекта программного средства	4
2.2 Неформальная постановка задачи.....	4
2.3 Формальная постановка задачи	9
2.4 Алгоритм решения задачи	15
2.5 Спецификация данных	17
2.6 Спецификация функций.....	18
2.7 Проект программного средства.....	20
2.8 Описание данных программы.....	21
2.9 Алгоритм программы на языке PDL	23
2.10 Тесты	25
2.11 Тестирование программы.....	28
3 Заключение	46
4 Список использованных источников	47

1 Введение

Учебно-технологическая (проектно-технологическая) практика проходила в вузе на базе департамента программной инженерии и искусственного интеллекта Института математики и компьютерных технологий ДВФУ во втором семестре 2025-2026 учебного года с 19.02.2026 г. по 04.07.2026 г. (в объеме 3 з.е., 108 часов).

Целями учебно-технологической (проектно-технологической) практики являются: приобретение первичных практических умений и навыков по разработке проектов программных систем и проектной документации, а также знакомство с профессиональными задачами, решаемыми при создании программных систем.

Задачами учебно-технологической (проектно-технологической) практики являются:

- сбор и анализ требований заказчика к программному продукту;
- формализация предметной области программного проекта по результатам технического задания и экспресс-обследования;
- участие в проектировании компонентов программного продукта в объеме, достаточном для их конструирования в рамках поставленного задания;
- создание компонент программного обеспечения (кодирование, отладка, модульное и интеграционное тестирование);
- разработка тестового окружения, создание тестовых сценариев;
- разработка и оформление эскизной, технической и рабочей проектной документации.

2 Основная часть

2.1 Описание проекта программного средства

Проект программного средства представляет собой математическую игру-тренажёр для поддержания и улучшения математических и логических способностей пользователя.

Проект разработан с использованием принципов нисходящего проектирования; алгоритм программы основан на принципе объектно-ориентированного программирования.

Для реализации проекта были использованы среда PyCharm Community Edition (предоставляющая удобные инструменты для отладки, рефакторинга и работы с несколькими модулями), язык программирования Python и кроссплатформенная событийно-ориентированная графическая библиотека Tkinter для создания графического интерфейса.

Работа выполнена командой из одного человека (Расторгуев Андрей Андреевич). Моя роль в команде по результатам теста Белбина – Универсальный солдат.

2.2 Неформальная постановка задачи

Цель – создать математическую игру-тренажёр, обеспечивающую бесконечную реиграбельность и устойчивый интерес пользователя за счёт вариантности контента.

Тематика задач – базовые разделы: арифметика, простые уравнения, проценты, числовые последовательности, геометрия (площади, теорема Пифагора) и другие темы школьной математики вплоть до 11 класса, а также олимпиадные задания.

Режимы игры

«Выживание» – сложность растёт не с номером этажа, а после каждого правильного ответа. Ошибка может снижать сложность. Игрок сам регулирует темп усложнения:

1. Параметр сложности (диапазон чисел, типы задач) увеличивается на 1 после каждого правильного ответа.
2. При ошибке сложность уменьшается на 1 (но не ниже начального значения).
3. Номер этажа в этом режиме может идти параллельно, но сложность привязана к счётчику правильных ответов подряд.

«На время» – на каждый ответ даётся ограниченное время (например, 30 секунд или 1 минута в зависимости от уровня сложности). Если игрок не успел – засчитывается ошибка (потеря жизни + спуск):

1. На интерфейсе появляется обратный таймер.
2. Если игрок не успел ввести ответ до окончания времени, это считается ошибкой: теряется жизнь, происходит спуск, и генерируется новая задача.
3. После правильного ответа таймер сбрасывается до полного значения.

«Практика» – ошибки не наказываются (не отнимаются жизни), нет спуска. Игрок просто решает задачи, а игра показывает правильный ответ при ошибке. Этажи растут только при правильных ответах.

Механика подъёма/падения

1. Игровой персонаж поднимается по уровням.
2. У игрока 3 жизни.
3. Правильный ответ – подъём на один этаж.
4. Неправильный ответ – потеря одной жизни и спуск на один этаж (но не ниже первого).

5. Жизни кончились – игра заканчивается, показывается достигнутый этаж, появляются кнопки «Новая игра», «Меню», «Выход» и отображается статистика (количество правильных/неправильных ответов в этой сессии).

Генерация заданий

1. Задачи генерируются процедурно с помощью функций, принимающих текущий уровень сложности.
2. Для упрощения проверки ответы должны быть целыми числами или десятичными дробями с фиксированной точностью.
3. Тип задачи выбирается случайным образом из набора, разрешённого для данного уровня.

Адаптивная сложность

Сложность зависит от номера этажа и выбранного режима (начальная, средняя, старшая школа). Пример: на начальных этажах числа до 10 и простые задачи на логику; на более высоких – до 100, появляются линейные уравнения, проценты; ещё выше – квадратные уравнения, геометрические задачи с теоремой Пифагора, теория вероятности. Чем выше, тем больше тем по математике начальных, средних и старших классов (в зависимости от выбранного режима).

Персонаж

1. Игровой персонаж – анимированный человечек, который перемещается по башне.
2. Персонаж отображается на этаже, на котором в текущий момент он выполняет задачу для перехода на этаж выше.
3. При подъёме персонаж «поднимается» на один этаж вверх (анимация движения), при спуске – перемещается вниз.

Структура окон программы

Стартовое окно («Начать игру»):

1. Содержит название игры, краткое описание.
2. Предоставляет выбор режима (Практика, На время, Выживание).
3. Выбор сложности, то есть выбор школы: начальная, средняя, старшая.
4. Кнопка «Начать» для перехода в игровое окно.
5. Кнопка «Выход» для выхода из игры.
6. Отображение сохранённого рекорда.

Игровое окно («Сама игра»):

1. Основное окно, где происходит игровой процесс.
2. Содержит все элементы интерфейса (этаж, задача, поле ввода, жизни, рекорд, таймер/сложность, башня и персонаж).
3. Кнопка «Меню» (возврат к стартовому окну).
4. Кнопка «Выход» (закрывает программу).
5. При окончании жизни автоматически переходит к окну результатов.

Окно результатов («Конец и результаты игры»)

1. Отображается после завершения игры (кончились жизни).
2. Показывает достигнутый этаж, количество правильных и неправильных ответов за сессию.
3. Содержит кнопку «Меню» (возврат к стартовому окну), кнопку Новая игра» (Запуск игры повторно, минуя стартовое окно и сохраняя сложность и режим) и кнопку «Выход» для закрытия программы.
4. Рекорд (максимальный этаж) обновляется и отображается на этом экране.

Визуализация уровня

1. На экране отображается башня, на заднем фоне – природа, горы.
2. Текущий этаж выделяется визуально (Контур вокруг этажа).

3. При правильном ответе – анимация подъёма (персонаж поднимается вверх).
4. При ошибке – анимация спуска (персонаж опускается вниз).

Интерфейс (обязательные элементы)

1. Номер текущего этажа.
2. Окошко с текстом задачи.
3. Поле ввода ответа и кнопка «Проверить».
4. Индикатор жизней (счётчик).
5. Отображение текущего рекорда (максимальный этаж за всё время) с обновлением в реальном времени.
6. Кнопка «Меню» для возвращения в меню.
7. Кнопка «Выход» для выхода из игры.

Интерфейс (дополнительные элементы по режимам)

1. Для режима «На время» – таймер.
2. Для режима «Выживание» – индикатор текущей сложности (счётчик).

Взаимодействие пользователя с игрой

1. Пользователь вводит число или цифру с клавиатуры.
2. Может отправить ответ либо нажав на Enter на клавиатуре, либо на кнопку «Проверить» на интерфейсе левой кнопкой мыши.

Обработка ввода

1. При некорректном вводе (не число, пустая строка) выводится пометка, что поле не должно быть пустым на экране; ошибкой это не считается, жизни не отнимаются.
2. Все задачи имеют ответ в виде числа (целого или десятичного).

Сохранение рекордов

1. Лучший результат сохраняется в текстовый файл в папке с программой.
2. При каждом новом рекорде файл обновляется.

2.3 Формальная постановка задачи

Входные данные:

$$A = \{a_i \mid a_i = \{question_i, answer_i, topic_i, difficulty_level_i\}, 1 \leq i \leq N\} \quad -$$

множество записей (база данных задач):

- $question_i = \{q_i^j \mid q_i^j \in \{'a'..'я', 'A'..'Я', '0'..'9', ' ', '!', '?', ':', ';', '(', ')', '-', '+', '/', '*'\}, 1 \leq j \leq 500\}$ – текст задачи (строка до 500 симв.);
- $answer_i \in \mathbb{R}$ – правильный ответ;
- $topic_i = \{t_i^j \mid t_i^j \in \{'a'..'я', 'A'..'Я', ' '\}, 1 \leq j \leq 30\}$ – тема задачи (строка до 30 симв.), $topic_i \in \{"арифметика", "уравнения", "проценты", "последовательности", "геометрия"\}$;
- $difficulty_level_i \in \mathbb{N}$, – уровень сложности.

$T_{max} = 30$ – максимальное время на ответ (сек);

$\varepsilon = 10^{-6}$ – допустимая абсолютная погрешность;

$H_{floor} \in \mathbb{Z}, H_{floor} > 0$ – высота одного этажа – высота одного этажа (пиксели);

$r_{saved} \in \mathbb{Z}, r_{saved} \geq 0$ – значение сохранённого рекорда, загружаемое из внешнего файла при старте программы;

$s = \{s^j \mid s^j \in \{'a'..'я', 'A'..'Я'\}, 1 \leq j \leq 15\}$ – выбранная школа (строка до 15 симв.), $s \in \{"Начальная", "Средняя", "Старшая"\}$;

$m = \{m^j \mid m^j \in \{'a'..'я', 'A'..'Я'\}, 1 \leq j \leq 15\}$ – режим игры (строка до 15 симв.), $m \in \{"Практика", "НаВремя", "Выживание"\}$;

$$d_{min}(s) = \begin{cases} 1, & \text{если } s = \text{"Начальная"} \\ 3, & \text{если } s = \text{"Средняя"} \\ 5, & \text{если } s = \text{"Старшая"} \end{cases};$$

$x \in R$ – числовой ответ, введённый пользователем с клавиатуры (или через поле ввода);

$cmd \in \{ "click_start", "click_submit", "click_new_game", "click_exit", "key_enter", "click_menu" \}$ – управляющая команда (нажатие кнопки мыши или клавиши);

$f_0 = 0$ – начальный этаж;

$l_0 = 3$ – начальное количество жизней;

$W_0 = "start"$ – начальное окно программы при запуске;

$char_init$ – параметры инициализации персонажа (спрайт, начальная позиция на экране);

$W_{screen} = 1920$ – ширина окна программы (пиксели);

$H_{screen} = 1080$ – высота окна программы (пиксели);

$E_{start} = \{ < title >, < desc >, < school >, < mode >, < start >, < exit >, < record > \}$ – набор UI-элементов стартового окна:

- $<title>$ – заголовок игры «Math Tower» (координаты (960,120), шрифт Arial 28, цвет фона lightblue);
- $<school_label>$ – метка «Выберите школу» (координаты (960,280), шрифт Arial 14);
- $<school>$ – группа радиокнопок выбора школы (координаты (960,320), шрифт Arial 12, варианты: «Начальная» (960,320), «Средняя» (960,350), «Старшая» (960,380));
- $<mode_label>$ – метка «Выберите режим:» (координаты (960,430), шрифт Arial 14);
- $<mode>$ – группа радиокнопок выбора режима (координаты (960,470), шрифт Arial 12, варианты: «Практика» (960,470), «НаВремя» (960,500), «Выживание» (960,530));
- $<start>$ – кнопка «Старт» (координаты (960,590), размеры 100×40, шрифт Arial 14, цвет фона green);

- `<exit>` – кнопка «Выход» (координаты (960,650), размеры 80×30, шрифт Arial 12);

$x_p = 90$ – фиксированная горизонтальная координата спрайта на Canvas (пиксели);

$H_{floor} = 50$ – высота одного этажа (пиксели).

Выходные данные:

$f' \in \mathbb{Z}, f' \geq 0$ – достигнутый этаж;

$l' \in \mathbb{Z}, 0 \leq l' \leq 3$ – оставшиеся жизни;

$r' \in \mathbb{Z}, r' \geq 0$ – обновлённый рекорд;

$d' \in \mathbb{Z}, d' \geq d_{min}(s)$ – новый уровень сложности;

$\tau' \in \mathbb{R}, \tau' \geq 0$ – оставшееся время (для режима «НаВремя»);

$c \in \{"correct", "wrong", "timeout"\}$ – статус ответа;

$correct_count' \in \mathbb{Z}, correct_count' \geq 0$ – количество правильных ответов;

$wrong_count' \in \mathbb{Z}, wrong_count' \geq 0$ – количество неправильных ответов;

$y_{char}' \in \mathbb{Z}$ – вертикальная координата спрайта персонажа (пиксели);

$offset' \in \mathbb{Z}$ – вертикальный сдвиг фонового изображения (пиксели).

$W' \in \{"start", "game", "results", "exit"\}$ – текущее отображаемое окно игры;

$E_{game} = \{< tower_canvas >, < floor_label >, < lives_label >, < record_label >, < difficulty_label >, < timer_label >, < task_frame >, < task_label >, < input_entry >, < check_btn >, < new_game_btn >, < exit_game_btn >, < sprite >\}$ – набор UI-элементов игрового окна:

- `tower_canvas` – Canvas с башней (координаты (100,50), размеры 180×600);
- `floor_label` – индикатор этажа (координаты (400,20), текст «Этаж: $<f'>$ »);
- `lives_label` – индикатор жизней (координаты (580,20), текст «Жизни: $<l'>$ »);

- `difficulty_label` – индикатор сложности (координаты (980,20), текст «Сложность: $\langle d' \rangle$ »);
- `timer_label` – таймер (координаты (1180,20), текст «Время: $\langle \tau' \rangle$ », виден при `m="НаВремя"`);
- `task_frame` – фрейм задачи (координаты (400,80), размеры 1200×600);
- `task_label` – текст задачи (координаты (1000,380), шрифт Arial 24);
- `input_entry` – поле ввода (координаты (400,720), размеры 250×35);
- `check_btn` – кнопка «Проверить» (координаты (670,720), размеры 120×40);
- `new_game_btn` – кнопка «Новая игра» (координаты (400,780), размеры 120×35);
- `exit_game_btn` – кнопка «Выход» (координаты (540,780), размеры 80×35);
- `sprite` – спрайт персонажа (координаты (90, `y_char'`) на Canvas).

$E_{res} = \{ \langle title_res \rangle, \langle final_floor \rangle, \langle correct \rangle, \langle wrong \rangle, \langle new_game_res \rangle, \langle exit_res \rangle \}$ – набор UI-элементов окна результатов:

- `<title_res>` – заголовок «Игра окончена» (координаты (960,150), шрифт Arial 28);
- `<final_floor>` – итоговый этаж (координаты (960,220), текст «Достигнутый этаж: $\langle f' \rangle$ »);
- `<correct>` – счётчик правильных (координаты (960,260), текст «Правильных ответов: $\langle correct_count' \rangle$ »);
- `<wrong>` – счётчик ошибок (координаты (960,300), текст «Неправильных ответов: $\langle wrong_count' \rangle$ »);
- `<new_game_res>` – кнопка «Новая игра» (координаты (960,380), размеры 140×40);
- `<exit_res>` – кнопка «Выход» (координаты (960,440), размеры 80×30).

Формализованные связи:

$f_0 = 0$, $l_0 = 3$, $r_0 = r_{saved}$, $d_0 = d_{min}(s)$, $\tau_0 = T_{max}$, $correct_count = 0$,
 $wrong_count = 0$;

$T = getRandomTask(d)$ – выбор случайной задачи сложности d ;

$isCorrect(x, T) = \begin{cases} 1, & \text{если } |x - answer(T)| < \varepsilon; \\ 0, & \text{иначе} \end{cases}$;

$c = \begin{cases} "timeout", & \text{если } m = "НаВремя" \text{ и } \tau \leq 0 \\ "correct", & \text{если } isCorrect(x, T) = 1 \\ "wrong", & \text{иначе} \end{cases}$;

Для $\forall c$:

- если $c = "correct"$, то $f' = f + 1$, $correct_count' = correct_count + 1$;
- если $c \in \{"wrong", "timeout"\}$, то $f' = \max(f - 1, 0)$, $wrong_count' = wrong_count + 1$.

Для $\forall c$ и $\forall m$:

- если $c = "correct"$, то $l' = l$;
- если $m = "Практика"$ и $c \neq "correct"$, то $l' = l$;
- если $m \neq "Практика"$ и $c \neq "correct"$, то $l' = l - 1$.
- если $m = "Выживание"$ и $c = "correct"$, то $d' = d + 1$;
- если $m = "Выживание"$ и $c \neq "correct"$, то $d' = \max(d - 1, d_{min}(s))$;
- если $m \neq "Выживание"$, то $d' = d$;
- если $m = "НаВремя"$, то $\tau' = T_{max}$;
- если $m \neq "НаВремя"$, то $\tau' = \tau$.

$r' = \max(r, f')$;

$y_{char}' = 600 - (f' + 1) * H_{floor} + 25$;

$offset' = -f * H_{floor}$;

Для E_{game} :

- $\langle floor_label \rangle = "Этаж: " + f$;
- $\langle lives_label \rangle = "Жизни: " + l$;
- $\langle difficulty_label \rangle = "Сложность: " + d$, $\langle difficulty_label \rangle \Leftrightarrow m = "Выживание"$;

- $\langle \text{timer_label} \rangle = \text{"Время: " + } \tau, \langle \text{timer_label} \rangle \Leftrightarrow m = \text{"НаВремя"};$
- $\langle \text{task_label} \rangle = \text{question}(T);$
- $\langle \text{final_floor} \rangle = \text{"Достигнутый этаж: " + } f;$
- $\langle \text{correct} \rangle = \text{"Правильных ответов: " + } \text{correct_count};$
- $\langle \text{wrong} \rangle = \text{"Неправильных ответов: " + } \text{wrong_count}.$

$\text{prepare}(\text{win})$ – открывает окно win;

exit – выход из программы;

Переходы между окнами:

$(W = \text{"start"}) \xrightarrow{\text{cmd}=\text{"click_start"}} \text{prepare}(\text{"game"});$

$(W = \text{"game"}) \xrightarrow{l'=0} \text{prepare}(\text{"results"});$

$(W = \text{"game"}) \xrightarrow{\text{cmd}=\text{"click_menu"}} \text{prepare}(\text{"start"});$

$(W = \text{"results"}) \xrightarrow{\text{cmd}=\text{"click_new_game"}} \text{prepare}(\text{"game"});$

$(W = \text{"results"}) \xrightarrow{\text{cmd}=\text{"click_menu"}} \text{prepare}(\text{"start"});$

$(W = \text{"start"}) \xrightarrow{\text{cmd}=\text{"click_exit"}} \text{exit};$

$(W = \text{"game"}) \xrightarrow{\text{cmd}=\text{"click_exit"}} \text{exit};$

$(W = \text{"results"}) \xrightarrow{\text{cmd}=\text{"click_exit"}} \text{exit}.$

Условия победы и проигрыша:

$(l' = 0) \xrightarrow{\text{Проигрыш}} \text{prepare}(\text{"results"});$

$$(f' > r) \xrightarrow{\text{Победа (Новый рекорд)}} (r' = f').$$

2.4 Алгоритм решения задачи

1. Начало.
2. Загрузить из файла best.txt сохранённый рекорд r_{saved} , присвоить $r = r_{saved}$.
3. Вывести стартовое окно с выбором школы s и режима m .
4. Выбор школы (кнопки «Начальная», «Средняя», «Старшая») и режима (кнопки «Практика», «НаВремя», «Выживание»).
5. Если пользователь ничего не выбрал:
 - 5.1. Сохраняются базовые настройки: начальная школа, режим «Практика».
6. Если нажата кнопка «Выход»:
 - 6.1. Перейти в п. 14.
7. Дождаться нажатия кнопки «Старт».
8. Присвоить начальные значения:
 - 8.1. $f = 0$, $l = 3$, $d = d_{min}(s)$, $correct_count = 0$, $wrong_count = 0$.
 - 8.2. $y_{char} = 0$, $offset = 0$ (начальная позиция).
 - 8.3. Если $m = \text{"НаВремя"}$:
 - 8.3.1. $\tau = 30$.
 - 8.3.2. Иначе $\tau = \infty$.
9. Переход к игровому окну.
10. Запустить игровой цикл, пока $l > 0$:
 - 10.1. Сгенерировать случайную задачу $T = getRandomTask(d)$ и вывести её на экран.
 - 10.2. Если нажата кнопка «Меню»:
 - 10.2.1. Перейти в п. 3.

- 10.3. Если нажата кнопка «Выход»:
 - 10.3.1. Перейти в п. 14.
- 10.4. Если $m = \text{"НаВремя"}$:
 - 10.4.1. Запустить обратный отсчёт τ .
 - 10.4.2. Дождаться ввода ответа x или истечения τ .
 - 10.4.3. Если $\tau \leq 0$:
 - 10.4.3.1. Присвоить $c = \text{timeout}$.
 - 10.4.4. Иначе если $|x - \text{answer}(T)| < \varepsilon$:
 - 10.4.4.1. Присвоить $c = \text{correct}$.
 - 10.4.5. Иначе $c = \text{wrong}$.
- 10.5. Иначе дождаться ввода x .
- 10.6. Если $c = \text{correct}$:
 - 10.6.1. $f = f + 1$.
 - 10.6.2. $\text{correct_count} = \text{correct_count} + 1$.
 - 10.6.3. Воспроизвести `correct.wav`, анимация подъёма персонажа, сдвиг фона на $-H$.
 - 10.6.4. Если $f > r$:
 - 10.6.4.1. $r = f$.
 - 10.6.4.2. Записать r в `best.txt`.
 - 10.6.5. Если $m = \text{Выживание}$:
 - 10.6.5.1. $d = d + 1$.
- 10.7. Если $c = \text{wrong}$ или $c = \text{timeout}$:
 - 10.7.1. $f = \max(f - 1, 0)$.
 - 10.7.2. $\text{wrong_count} = \text{wrong_count} + 1$.
 - 10.7.3. Воспроизвести `wrong.wav`, анимация спуска персонажа, сдвиг фона на $+H$.
 - 10.7.4. Если $m \neq \text{"Практика"}$:
 - 10.7.4.1. $l = l - 1$.

10.7.5. Если $m = \text{"Выживание"}$:

10.7.5.1. $d = \max(d - 1, d_{\min}(s))$.

10.8. Если $m = \text{"НаВремя"}$:

10.8.1. $\tau = 30$.

11. Переход к окну результатов.

12. Вывести: $f, correct_count, wrong_count$.

13. Ожидание выбора пользователя:

13.1. Если нажата кнопка «Новая игра»:

13.1.1. Перейти в п. 3.

13.2. Если нажата кнопка «Выход»:

13.2.1. Перейти в п. 14.

14. Конец.

2.5 Спецификация данных

Входные данные:

best.txt – текстовый файл, содержащий 1 строку – сохранённый рекорд (максимальный достигнутый этаж). Формат данных:

<record>

Ввод осуществляется с клавиатуры:

- <answer> – числовой ответ на задачу.

Ввод осуществляется левой кнопкой мыши (ЛКМ) в Игровом окне и Окне результатов:

- <check> – кнопка «Проверить» для отправки и проверки ответа;
- <new_game> – кнопка «Новая игра» для перезапуска сессии;
- <menu> – кнопка «Меню» для возврата в Стартовое окно.

Выходные данные:

best.txt (обновляемый) – текстовый файл для сохранения нового рекорда. Формат данных:

<new_record>

Вывод осуществляется в графический интерфейс программы:

Игровое окно:

- <floor_label> – номер текущего этажа (целое число);
- <lives_label> – количество оставшихся жизней (визуальный индикатор, целое от 0 до 3);
- <task_label> – текст сгенерированной математической задачи;
- <timer_label> – оставшееся время в секундах (для режима «НаВремя»);
- <difficulty_label> – текущий уровень сложности (индикатор для режима «Выживание»);
- <sprite> – вертикальная координата спрайта персонажа (пиксели).

Окно результатов:

- <final_floor> – итоговый достигнутый этаж;
- <correct> – количество правильных ответов за сессию;
- <wrong> – количество неправильных ответов за сессию.

Сообщения пользователю (вывод на экран):

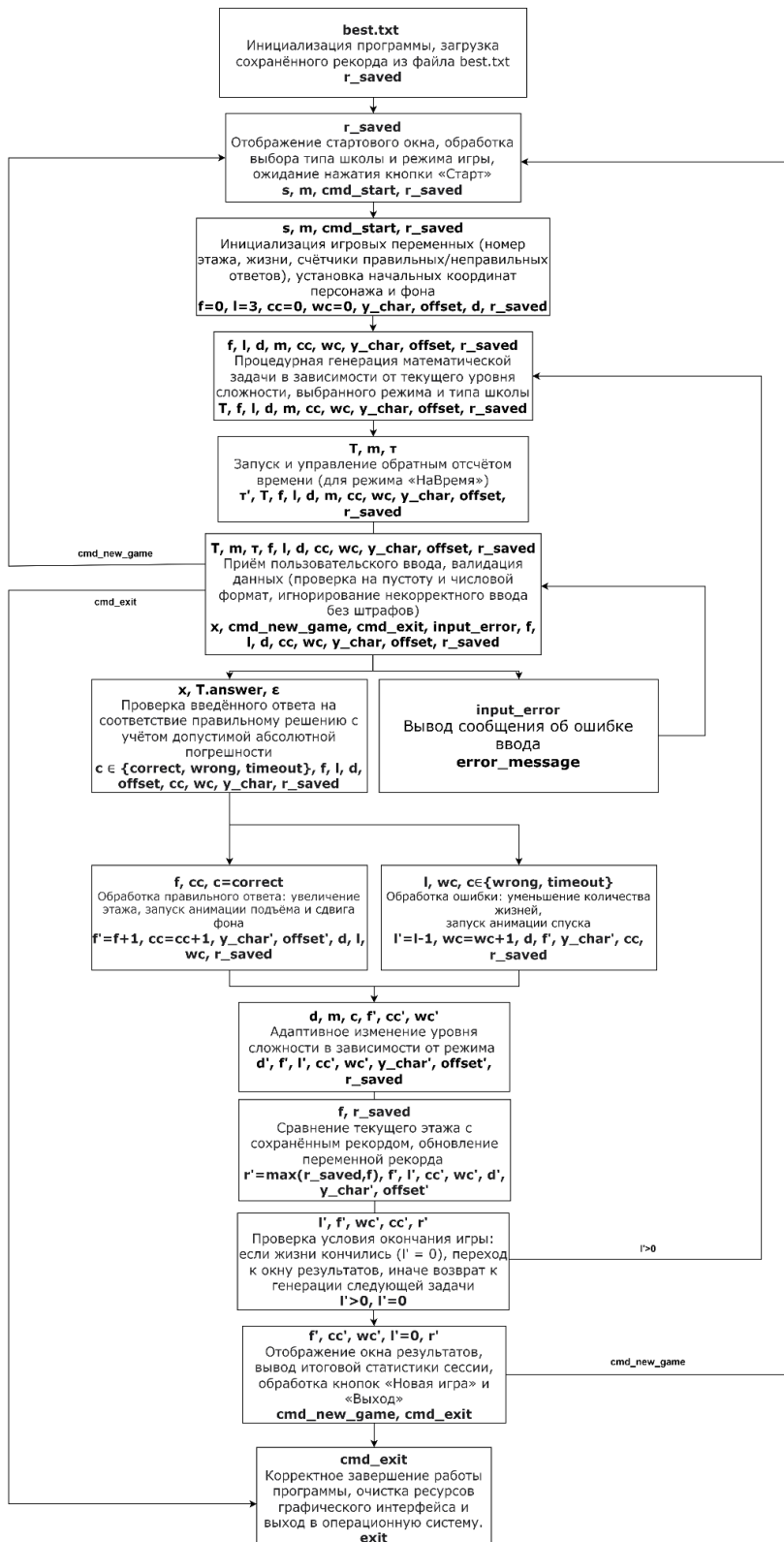
- <msg_input_error> – «Введите число» (при некорректном вводе).

2.6 Спецификация функций

1. Инициализация программы, загрузка сохранённого рекорда из файла best.txt.
2. Отображение стартового окна, обработка выбора типа школы и режима игры, ожидание нажатия кнопки «Старт».

3. Инициализация игровых переменных (номер этажа, жизни, счётчики правильных/неправильных ответов), установка начальных координат персонажа и фона.
4. Процедурная генерация математической задачи в зависимости от текущего уровня сложности, выбранного режима и типа школы.
5. Запуск и управление обратным отсчётом времени (для режима «НаВремя»), обработка истечения таймера как ошибки.
6. Приём пользовательского ввода, валидация данных (проверка на пустоту и числовой формат, игнорирование некорректного ввода без штрафов).
7. Проверка введённого ответа на соответствие правильному решению с учётом допустимой абсолютной погрешности.
8. Обработка правильного ответа: увеличение номера этажа, запуск анимации подъёма персонажа и сдвига фона.
9. Обработка ошибки: уменьшение количества жизней, запуск анимации спуска.
10. Адаптивное изменение уровня сложности в зависимости от режима.
11. Отображение окна результатов, вывод итоговой статистики сессии, обработка кнопок «Новая игра» и «Выход».
12. Вывод сообщения об ошибке ввода.
13. Корректное завершение работы программы, очистка ресурсов графического интерфейса и выход в операционную систему.
14. Проверка условия окончания игры: если жизни кончились ($l' = 0$), переход к окну результатов, иначе возврат к генерации следующей задачи.
15. Сравнение текущего этажа с сохранённым рекордом, обновление переменной рекорда.

2.7 Проект программного средства



2.8 Описание данных программы

Таблица 1 – Описание данных программы

Идентификатор	Назначение	Тип данных
root	Корневое окно приложения Tkinter	tk.Tk
T_MAX	Максимальное время на ответ в режиме «НаВремя»	int
EPS	Допустимая абсолютная погрешность проверки ответа	float
H	Высота одного этажа башни в пикселях	int
school	Выбранный тип школы («Начальная», «Средняя», «Старшая»)	str
mode	Выбранный режим игры («Практика», «НаВремя», «Выживание»)	str
record	Сохранённый рекорд (максимальный достигнутый этаж)	int
current_floor (f)	Текущий этаж игрока	int
lives (l)	Количество оставшихся жизней	int
difficulty (d)	Текущий уровень сложности задач	int
timer_value (τ)	Оставшееся время на ответ	float
correct_count	Счётчик правильных ответов за текущую сессию	int
wrong_count	Счётчик неправильных ответов за текущую сессию	int
tasks_db (A)	База данных процедурно генерируемых задач	list[tuple]
current_task (T)	Текущая задача (кортеж: текст, ответ, сложность)	tuple
timer_job	Идентификатор задачи таймера	str / None
best.txt	Внешний файл для хранения сохранённого рекорда	file
Локальный контекст процедуры load_record:		
f	Файловый объект для чтения рекорда	file
Локальный контекст процедуры save_record:		
f	Файловый объект для записи рекорда	file
record	Значение рекорда для сохранения	int
Локальный контекст процедуры generate_tasks_db:		
tasks	Временный список для накопления задач	list
i, j	Счетчики циклов генерации арифметики	int
a, b	Счетчики циклов генерации уравнений	int

p	Счетчик цикла генерации процентов	int
num	Число для расчета процентов	int
Локальный контекст процедуры get_random_task:		
difficulty	Требуемый уровень сложности	int
suitable	Список подходящих задач	list
Локальный контекст процедуры show_start_window:		
widget	Элемент интерфейса для очистки окна	tk.Widget
bg_image	Объект фоновое изображение	tk.PhotoImage
bg_label	Метка для отображения фона	tk.Label
schools	Список вариантов школ	list[tuple]
modes	Список вариантов режимов	list[str]
name, diff	Временные переменные для радиокнопок	str, int
m	Временная переменная режима	str
Локальный контекст процедуры show_game_window:		
main_frame	Основной фрейм игрового окна	tk.Frame
left_frame	Левая панель с башней	tk.Frame
right_frame	Правая панель с задачей	tk.Frame
info_frame	Фрейм информационной строки	tk.Frame
task_frame	Фрейм для текста задачи	tk.Frame
entry_frame	Фрейм поля ввода	tk.Frame
control_frame	Фрейм кнопок управления	tk.Frame
submit_btn	Кнопка «Проверить»	tk.Button
event	Событие нажатия Enter	tk.Event
Локальный контекст процедуры draw_tower:		
height	Высота Canvas	int
num_floors	Количество отрисовываемых этажей	int
floor_height	Высота одного прямоугольника этажа	int
i	Счетчик цикла отрисовки этажей	int
y	Вертикальная координата текущего этажа	int
char_y	Вертикальная координата персонажа	int
Локальный контекст процедуры update_tower:		
height	Высота Canvas	int

floor_height	Высота одного этажа	int
char_y	Новая вертикальная координата персонажа	int
Локальный контекст процедуры check_answer:		
user_answer (x)	Числовой ответ, введенный пользователем	float
correct	Правильный ответ из текущей задачи	float
Локальный контекст процедуры game_over:		
widget	Элемент интерфейса для очистки окна	tk.Widget

2.9 Фрагмент кода программы

```
def check_answer(self):
    """Проверка ответа пользователя и переход на следующий этаж"""
    try:
        # Получение ответа из поля ввода
        user_answer = float(self.answer_entry.get())
    except ValueError:
        # Обработка некорректного ввода
        messagebox.showerror("Ошибка", "Введите число!")
        return

    correct = self.current_task[1] # Правильный ответ из задачи

    # Проверка ответа с учётом погрешности ε
    if abs(user_answer - correct) < self.EPS:
        # ===== ПРАВИЛЬНЫЙ ОТВЕТ =====
        self.correct_count += 1      # Увеличение счётчика правильных
        self.current_floor += 1      # Подъём на этаж выше (f' = f + 1)

    # Проверка и обновление рекорда
    if self.current_floor > self.record:
        self.record = self.current_floor
        self.save_record(self.record)

    # Режим "Выживание": увеличение сложности
```

```

if self.mode == "Выживание":
    self.difficulty += 1

# Обновление интерфейса
self.update_ui_after_correct()

# Генерация новой задачи для следующего этажа
self.generate_new_task()

# Сброс таймера для режима "НаВремя"
if self.mode == "НаВремя":
    self.timer_value = self.T_MAX
    self.start_timer()
else:
    # ===== НЕПРАВИЛЬНЫЙ ОТВЕТ =====
    self.wrong_count += 1          # Увеличение счётчика ошибок

# Потеря жизни (кроме режима "Практика")
if self.mode != "Практика":
    self.lives -= 1              # l = l - 1

# Спуск на этаж ниже (но не ниже первого)
self.current_floor = max(self.current_floor - 1, 0)

# Режим "Выживание": снижение сложности
if self.mode == "Выживание":
    self.difficulty = max(self.difficulty - 1,
                          self.get_initial_difficulty())

# Обновление интерфейса
self.update_ui_after_wrong()

# Генерация новой задачи
self.generate_new_task()

```

```
# Сброс таймера
if self.mode == "НаВремя":
    self.timer_value = self.T_MAX
    self.start_timer()
```

```
# Очистка поля ввода
self.answer_entry.delete(0, tk.END)
```

```
# Проверка окончания игры (жизни кончились)
if self.lives == 0:
    self.game_over()
```

2.10 Тесты

Метод тестирования: белый ящик – покрытие ветвей.

Таблица 2 – Карта покрытия ветвей программы

№	Модуль / Метод	Условие в коде (Python)	True-ветвь (Результат выполнения)	False-ветвь (Результат выполнения)	№ (True)	№ (False)
1	load_record()	FileNotFoundError при открытии best.txt	Возврат 0 (инициализация нового рекорда)	Чтение целого числа из файла в переменную record	1	2
2	get_initial_diff iculty()	self.school == "Начальная"	Присваивание d = 1	Переход к проверке условия elif self.school == "Средняя"	3	4
3	get_initial_diff iculty()	self.school == "Средняя"	Присваивание d = 3	Присваивание d = 5 (для "Старшей" школы)	4	5
4	start_game()	self.mode == "НаВремя"	Инициализация self.timer_value =	Присваивание self.timer_value =	6	3

			T_MAX, запуск таймера	None (таймер не активен)		
5	check_answer()	ValueError при преобразован ии ввода в float	Вывод messagebox.showerr or("Ошибка", "Введите число!"), выход из метода	Продолжение логики проверки ответа	17	9
6	check_answer()	abs(user_ans wer - correct) < EPS	Переход к блоку обработки правильного ответа (f+=1)	Переход к блоку обработки неправильного ответа (l-=1)	9	10
7	check_answer() (блок успеха)	self.current_fl oor > self.record	Обновление self.record = self.current_floor, вызов save_record()	Рекорд не обновляется, файл не перезаписывается	12	6
8	check_answer() (блок успеха)	self.mode == "Выживание"	Увеличение сложности self.difficulty += 1	Сложность d остается неизменной	7	6
9	check_answer() (блок ошибки)	self.mode != "Практика"	Уменьшение жизней self.lives -= 1	Жизни l сохраняются (режим тренировки)	10	8
10	check_answer() (блок ошибки)	self.mode == "Выживание"	Снижение сложности d' = max(d-1, d_min)	Сложность d остается неизменной	7	10
11	process_timeo ut()	self.mode != "Практика"	Уменьшение жизней self.lives -= 1	Жизни l сохраняются	11	8
12	check_answer() / process_timeo ut()	self.lives == 0	Вызов метода self.game_over() (переход к окну результатов)	Продолжение игрового цикла (генерация новой задачи)	13	6

13	game_over()	Нажатие кнопки «Новая игра» (cmd_new_game)	Вызов self.show_start_window() (полный перезапуск с выбором настроек)	Проверка других кнопок интерфейса	15	16
14	game_over()	Нажатие кнопки «Выход» (cmd_exit)	Вызов self.root.quit() (закрытие приложения)	—	16	—

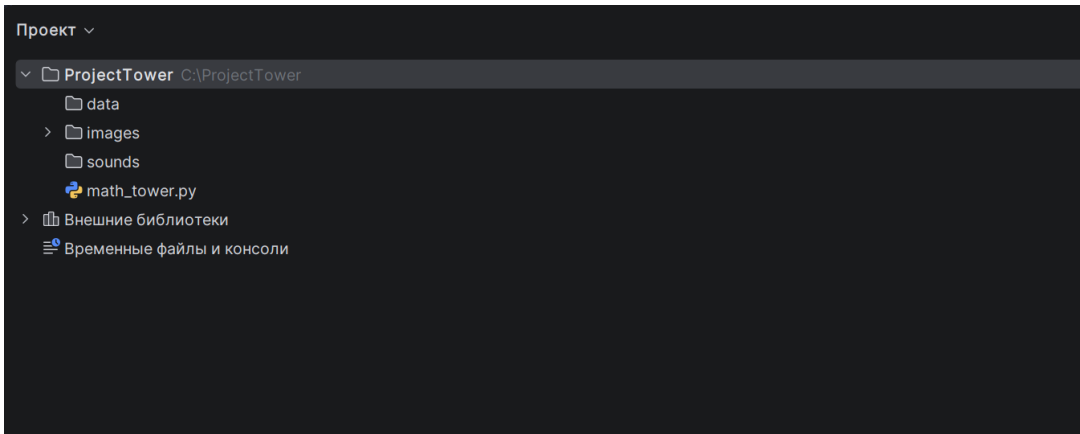
Таблица 3 – Тестовые сценарии

№	Входные данные	Выходные данные / Результат	Покрытые ветви
1	Файл best.txt отсутствует в директории проекта	record = 0, игра начинается с нулевого рекорда	1(F)
2	Файл best.txt содержит строку "42"	record = 42, отображается в стартовом окне	1(T)
3	Выбор: Школа="Начальная", Режим="Практика"	difficulty = 1, таймер не запущен	2(T), 4(F), 9(F)
4	Выбор: Школа="Средняя", Режим="Выживание"	difficulty = 3, отображается индикатор сложности	2(F), 3(T), 8(T), 10(T)
5	Выбор: Школа="Старшая", Режим="НаВремя"	difficulty = 5, таймер запущен со значением 30 сек	2(F), 3(F), 4(T)
6	Режим="НаВремя", введен правильный ответ за 15,1 сек	Таймер сброшен до 30, этаж увеличен (f++), жизни сохранены	4(T), 6(T), 7(F), 9(F)
7	Режим="Выживание", серия из 3 правильных ответов подряд	Сложность последовательно увеличивается: 1→2→3→4	8(T)
8	Режим="Практика", введен неверный ответ	Жизни не отнимаются (l=3), отображается подсказка/правильный ответ	9(F)
9	Задача: 7 - 2 = ?, Ввод: 5	Статус c="correct", подъем на этаж, персонаж поднялся на следующий этаж	6(T)

10	Задача: 1 минута = ? секунд, Ввод: 10	Статус c="wrong", спуск на этаж, потеря жизни (если не "Практика")	6(F)
11	Режим="НаВремя", ожидание истечения 30 секунд	Статус c="timeout", потеря жизни, спуск на этаж	11(T)
12	Текущий этаж f=45, сохраненный рекорд r=42	Рекорд обновлен r'=45, файл best.txt перезаписан	7(T)
13	Текущие жизни l=1, введен неверный ответ	Жизни l'=0, автоматический переход к окну результатов	12(T)
14	В игровом окне нажата кнопка «Меню»	Возврат в стартовое окно с сохранением текущих настроек	11(T)
15	В окне результатов нажата кнопка «Новая игра»	Сброс переменных f=0, l=3, ss=0, ws=0, начало новой сессии	13(T)
16	В любом окне нажата кнопка «Выход»	Корректное завершение работы приложения, очистка ресурсов	14(T)
17	В поле ввода ответа введено "abc" или оставлено пустым	Появление всплывающего окна messagebox с текстом «Введите число!»	5(T)

2.11 Тестирование программы

Таблица 4 – Тестирование программы

№	Результат
1	Покрыта ветвь 1(F)  Рисунок 1 – Отсутствие файла рекорда

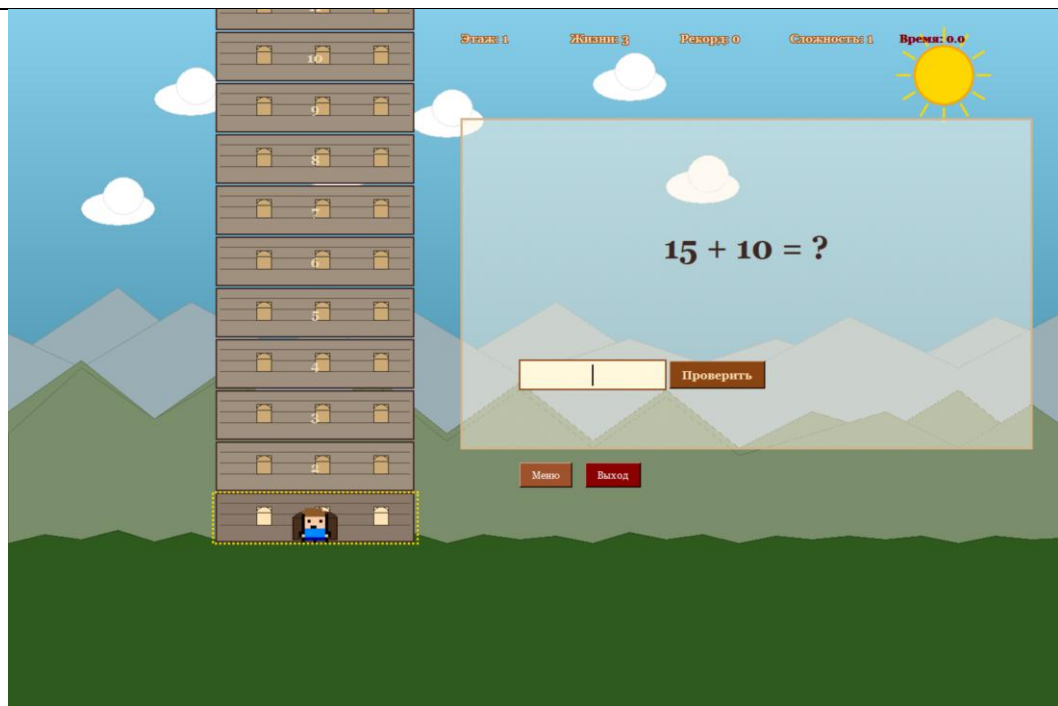


Рисунок 2 – Начало игры с рекордом 0

2

Покрыта ветвь 1(T)

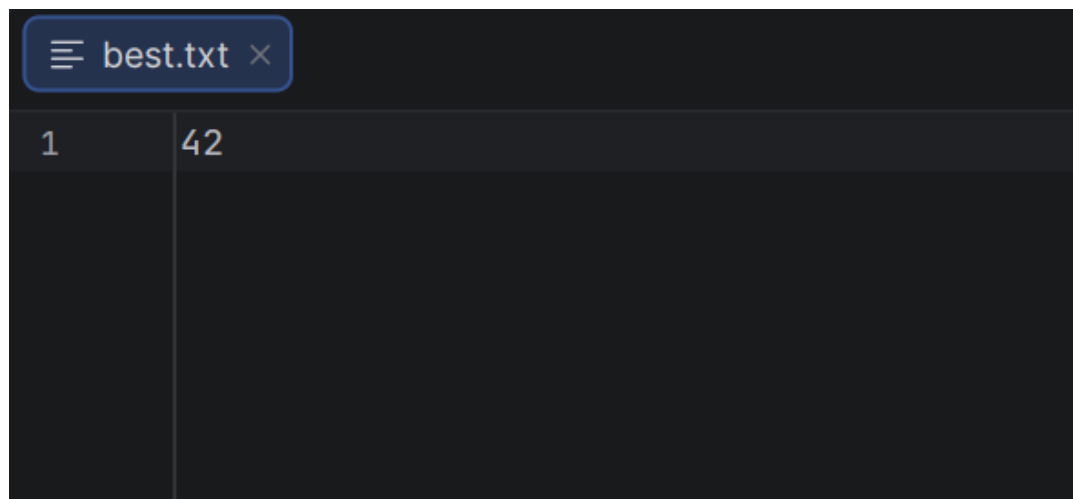


Рисунок 3 – Файл рекорда со строкой «42»

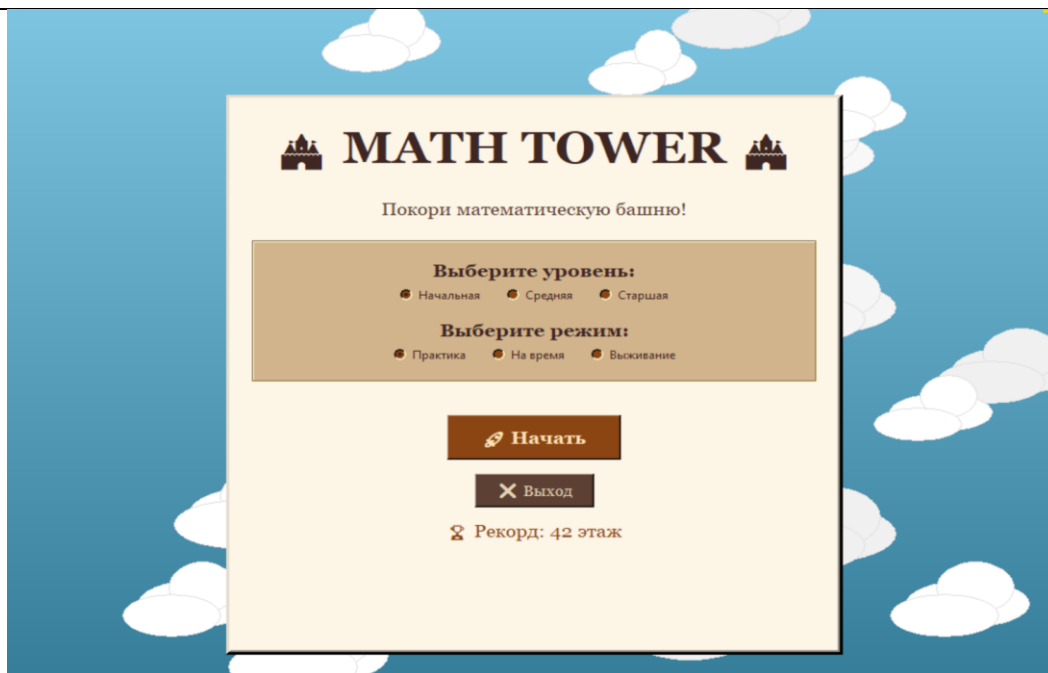


Рисунок 4 – Стартовое окно с рекордом 42

3

Покрыта ветвь 2(T), 4(F), 9(F)

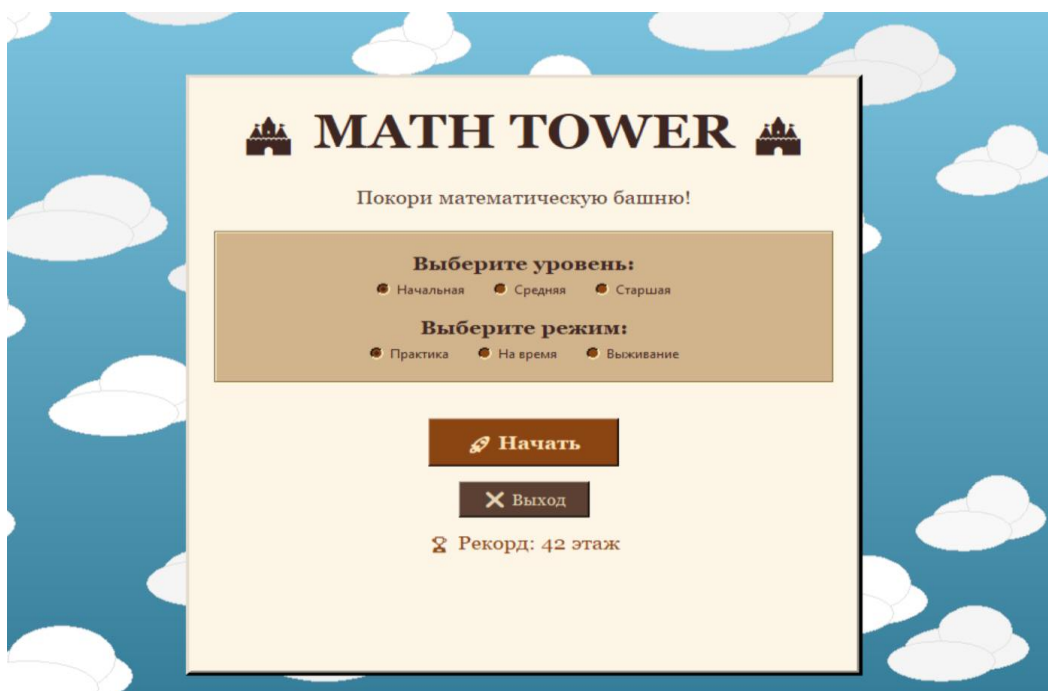


Рисунок 5 – Стартовое окно с уровнем «Начальная» и режимом «Практика»

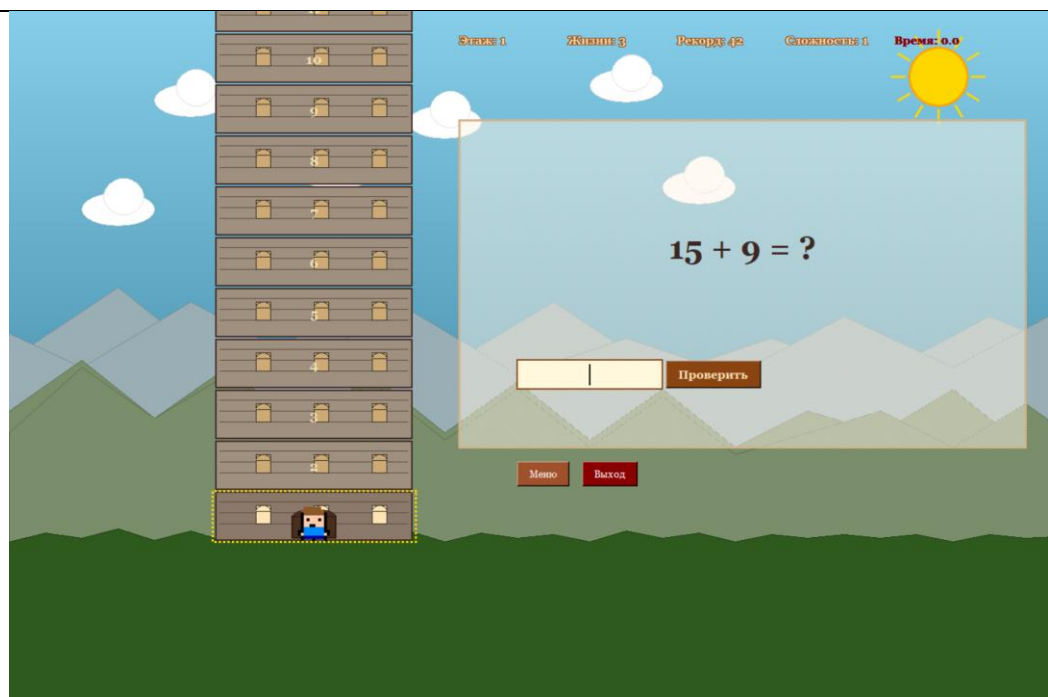


Рисунок 6 – Игровое окно с незапущенным таймером

4

Покрыты ветви 2(F), 3(T), 8(T), 10(T)

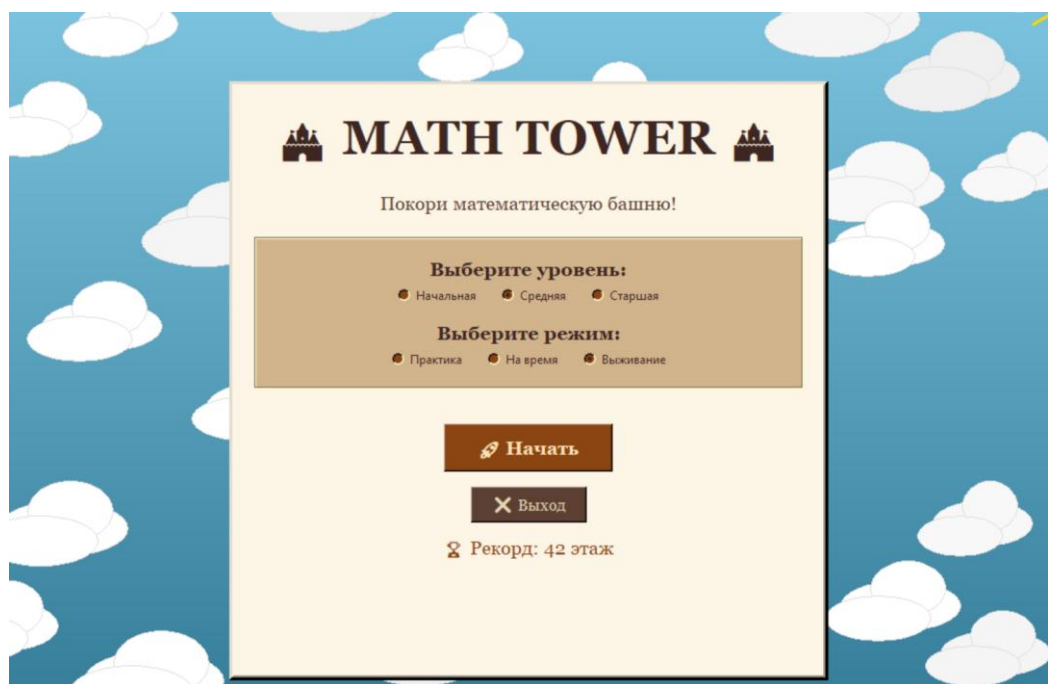


Рисунок 7 – Стартовое окно с уровнем «Средняя» и режимом «Выживание»

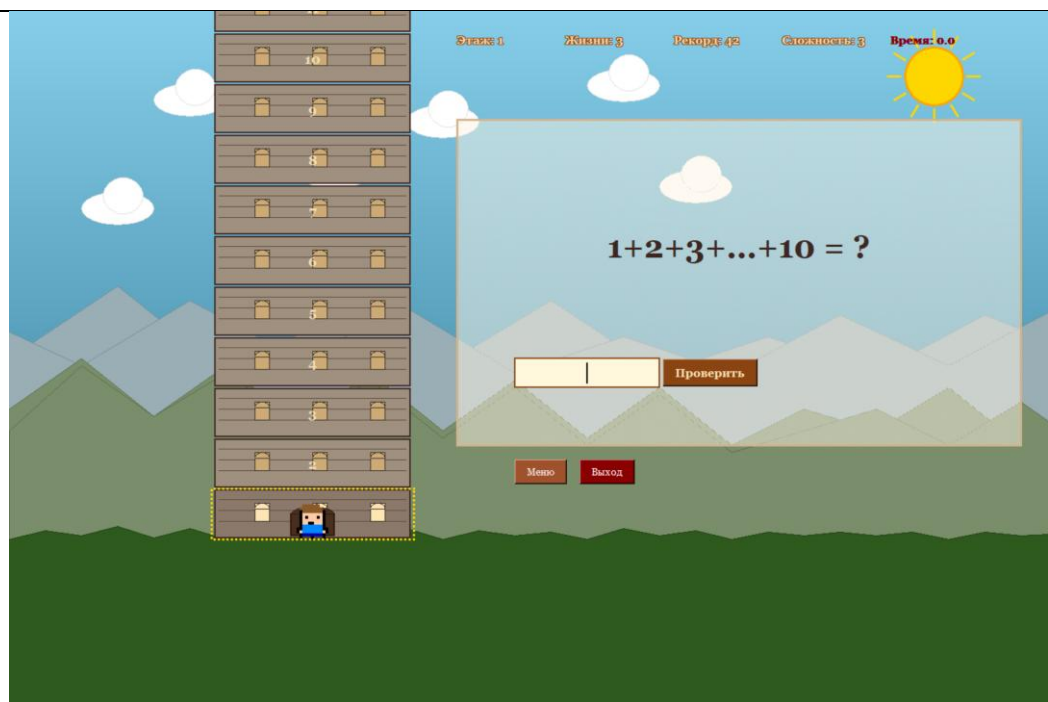


Рисунок 8 – Игровое окно с индикатором сложности 3

5

Покрыты ветви 2(F), 3(F), 4(T)

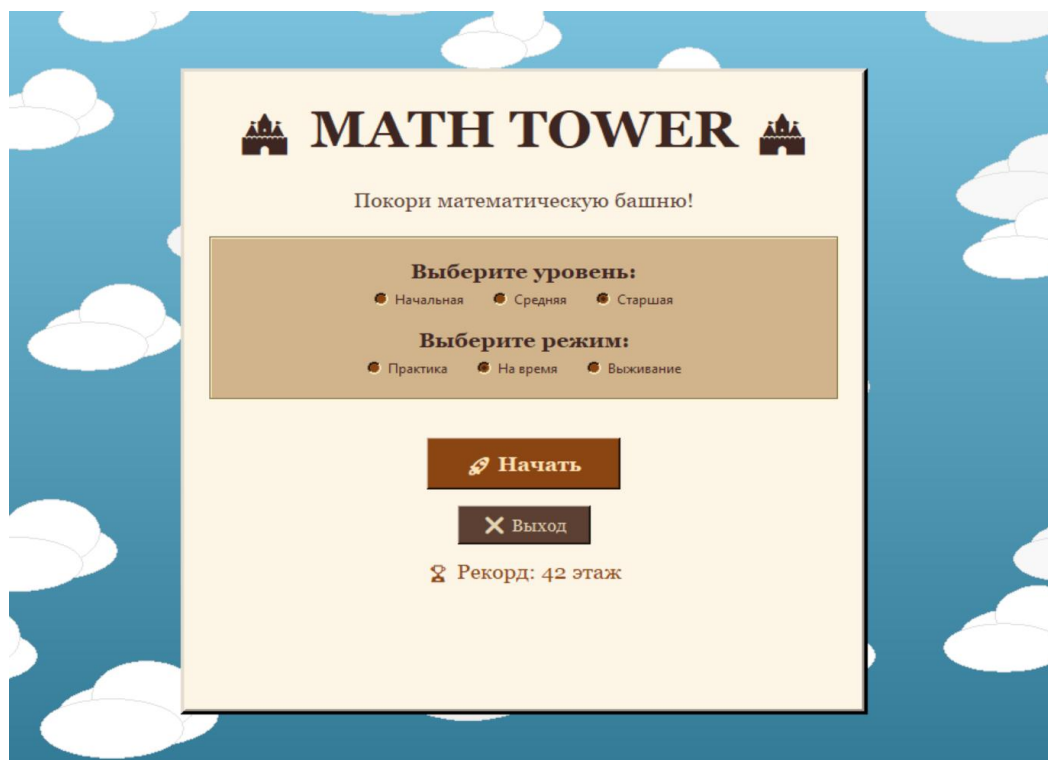


Рисунок 9 – Стартовое окно с уровнем «Старшая» и режимом «НаВремя»

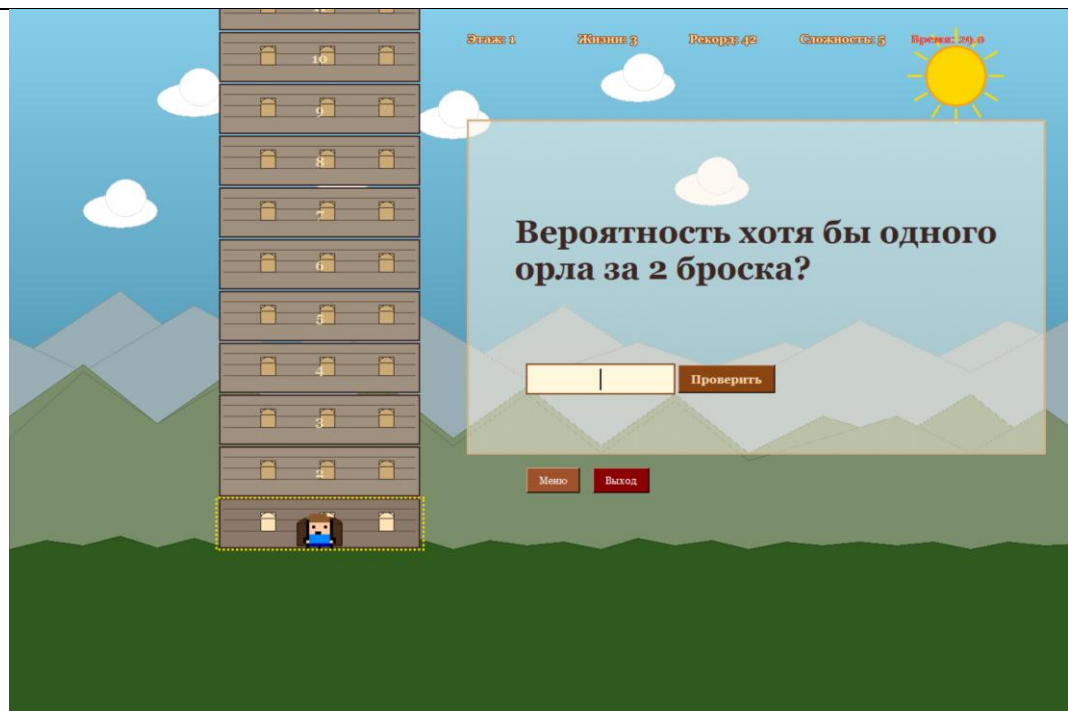


Рисунок 10 – Игровое окно с запущенным таймер

6 Покрывают ветви 4(T), 6(T), 7(F), 9(F)

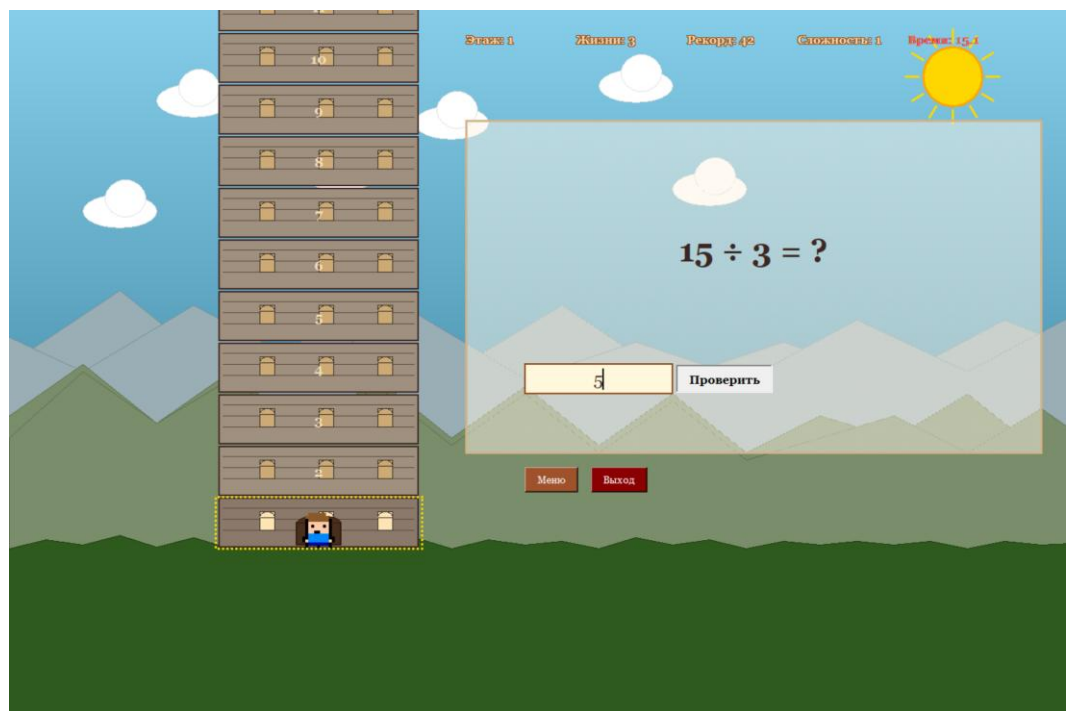


Рисунок 11 – Игровое поле с таймером 15.1

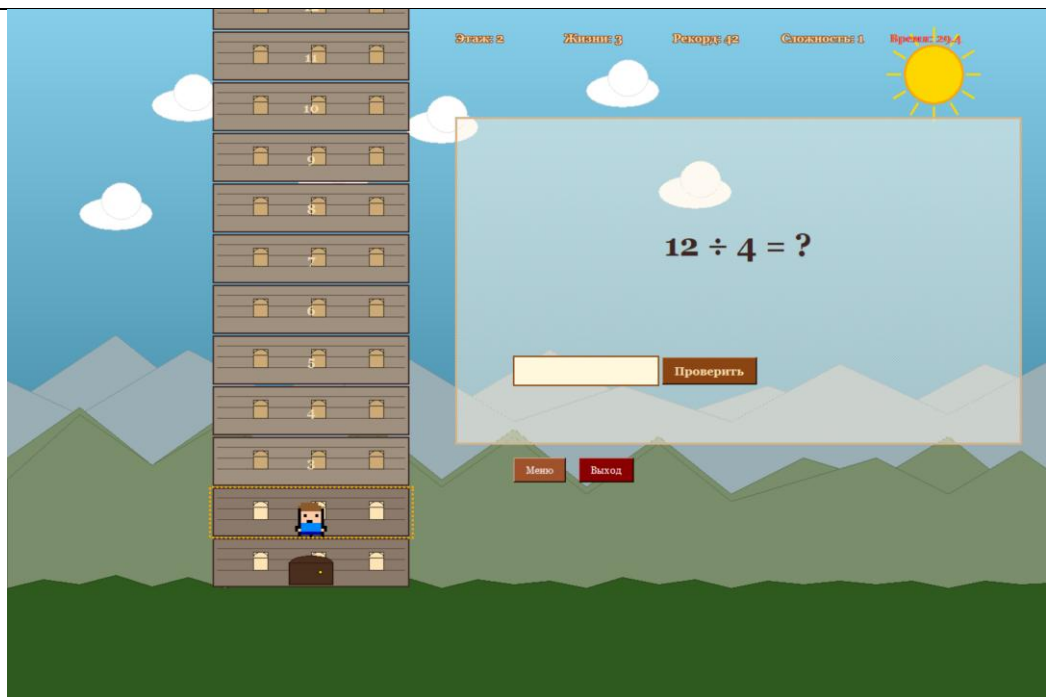


Рисунок 12 – Игровое поле с обновленным таймером после правильного ответа

7

Покрывает ветвь 8(T)

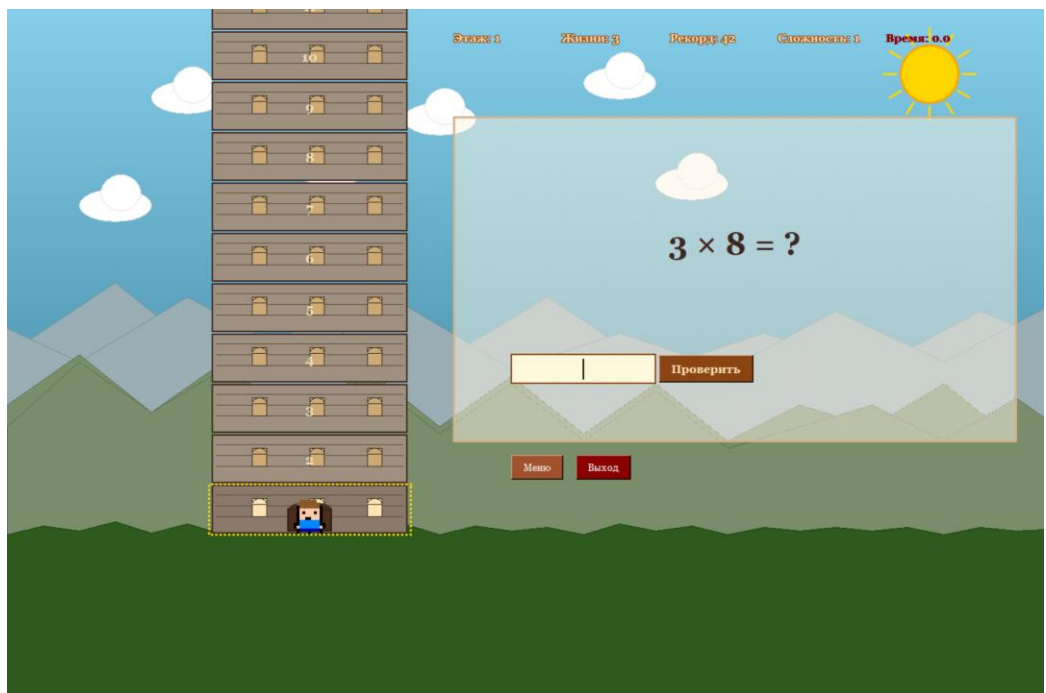


Рисунок 13 – Игровое поле со сложностью 1

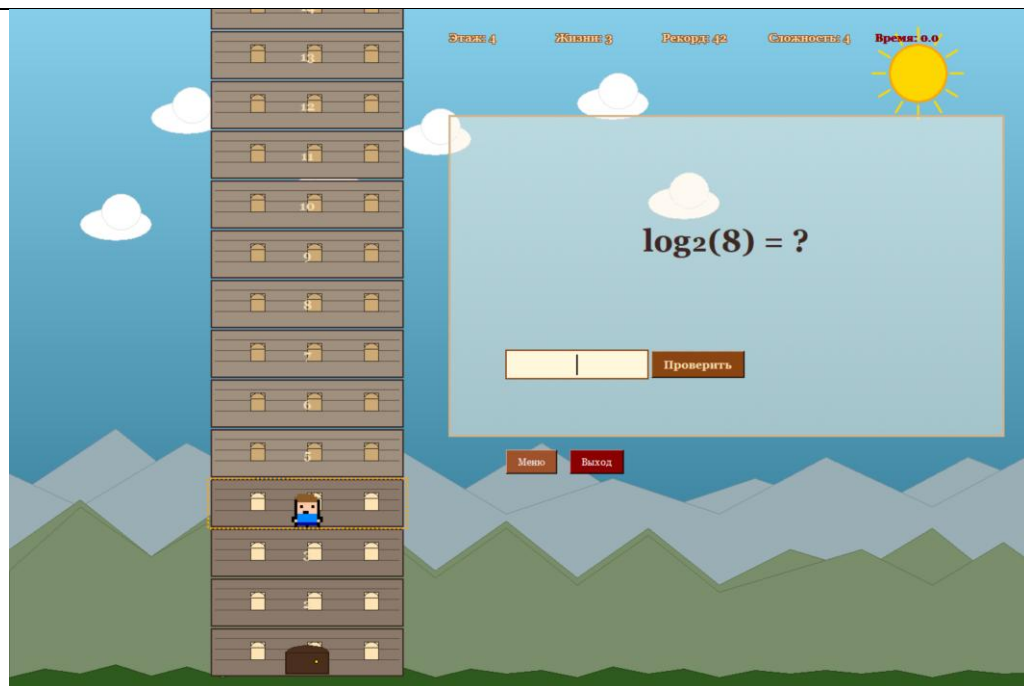


Рисунок 14 – Игровое окно со сложностью 4 после трёх правильных ответов

8

Покрыта ветвь 9(F)

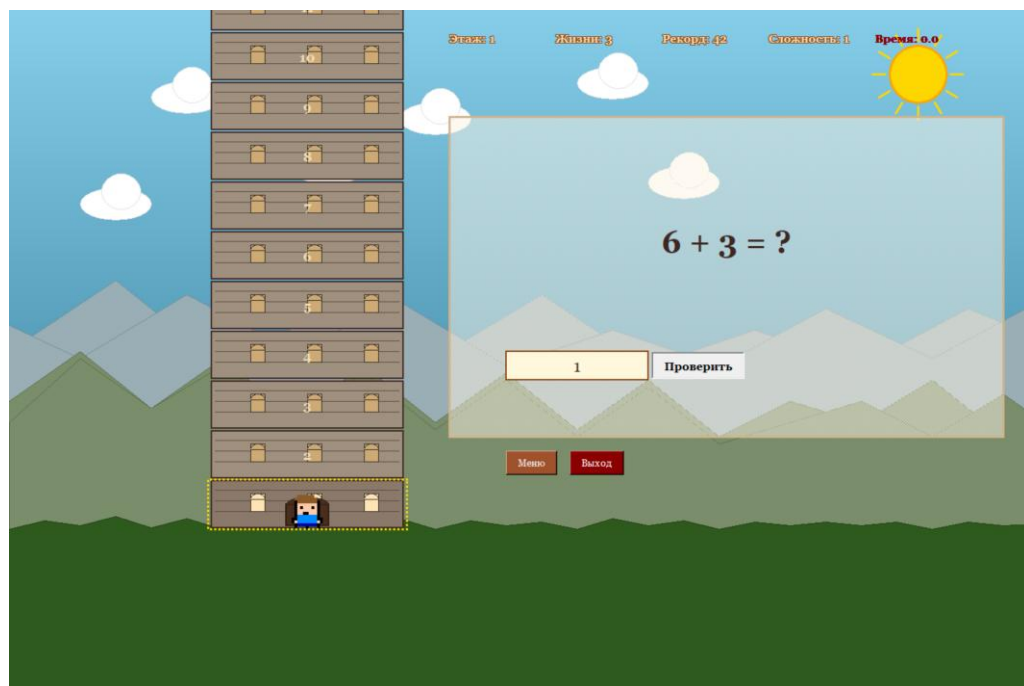


Рисунок 15 – Игровое окно с режимом «Практика» с неправильно введённым ответом

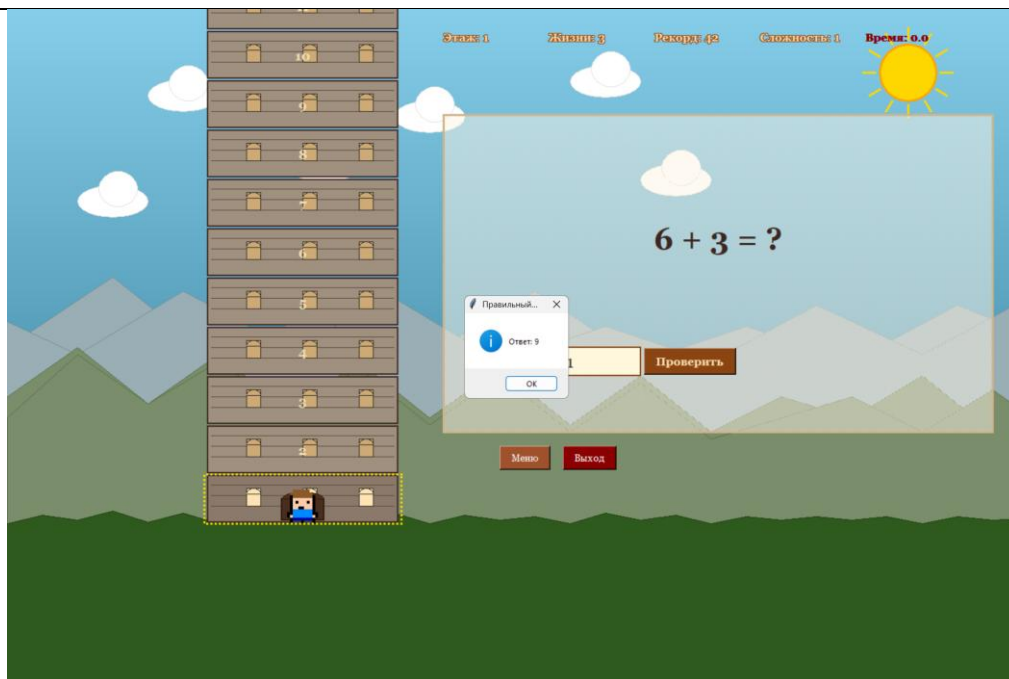


Рисунок 16 – Отображение подсказки после неправильного ответа

9

Покрыта ветвь 6(Т)

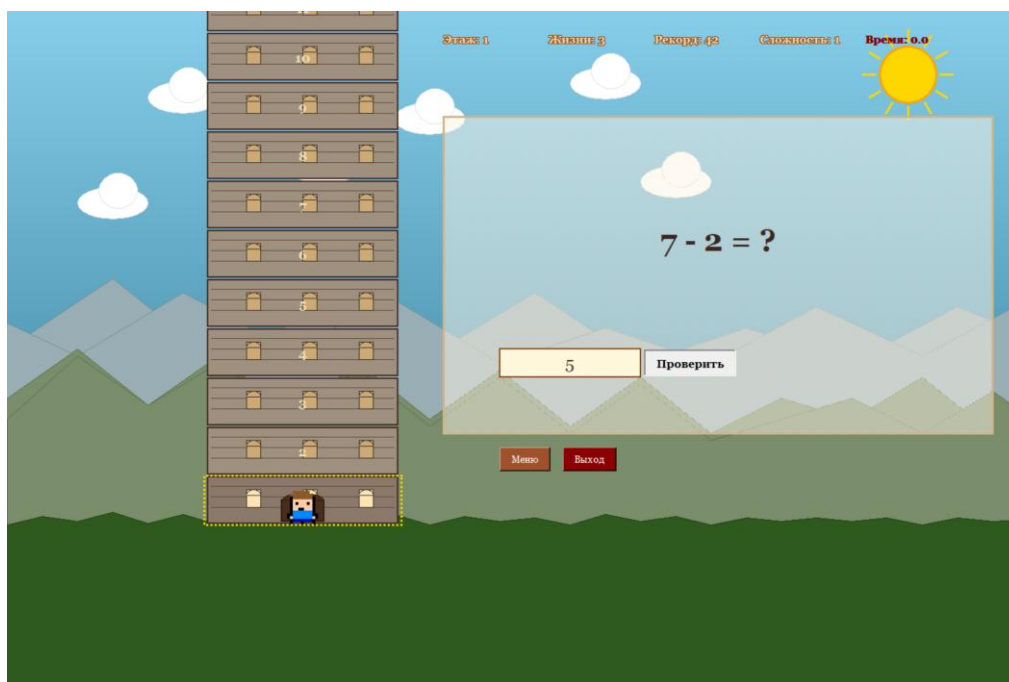


Рисунок 17 – Игровое окно с правильно введённым ответом

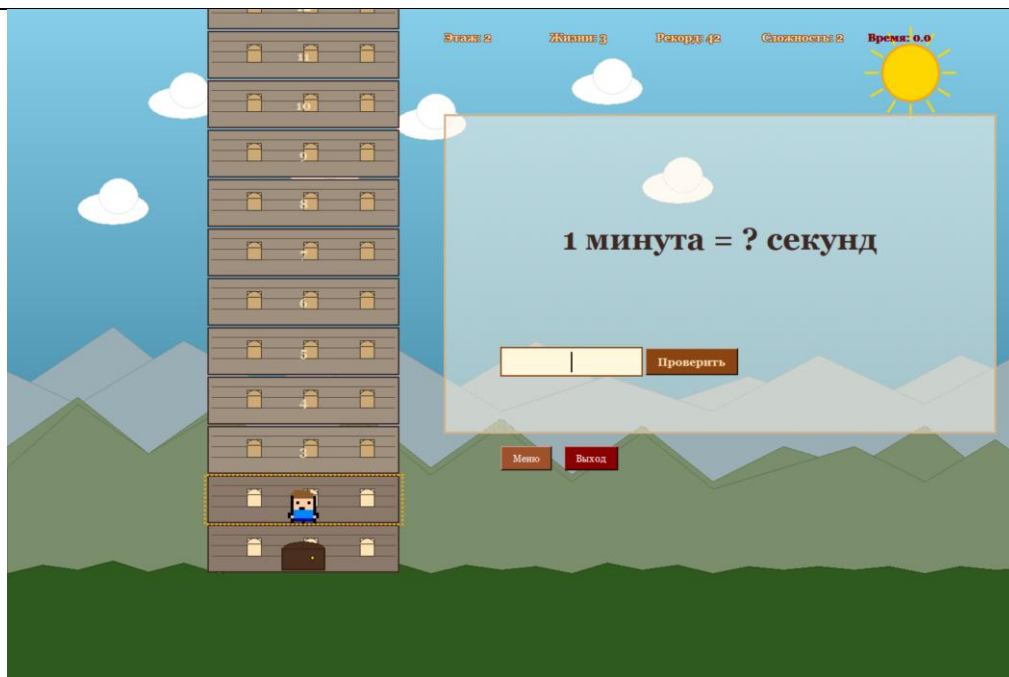


Рисунок 18 – Поднятие персонажа на этаж выше после правильного ответа

10

Покрыта ветвь 6(F)

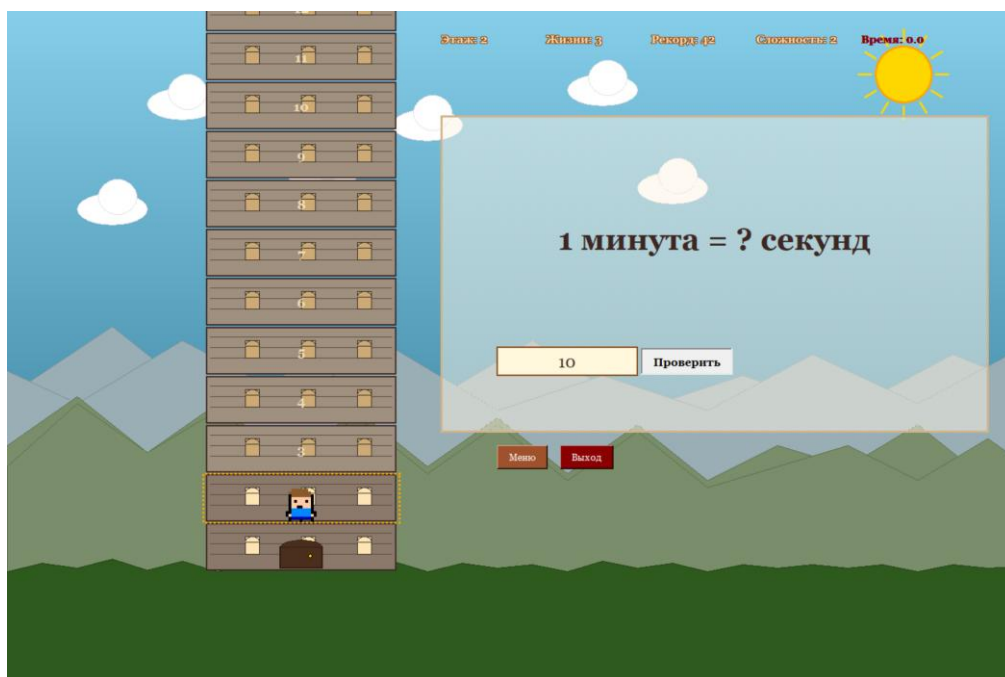


Рисунок 19 – Игровое окно с режимом «Выживание» с неправильно введённым ответом

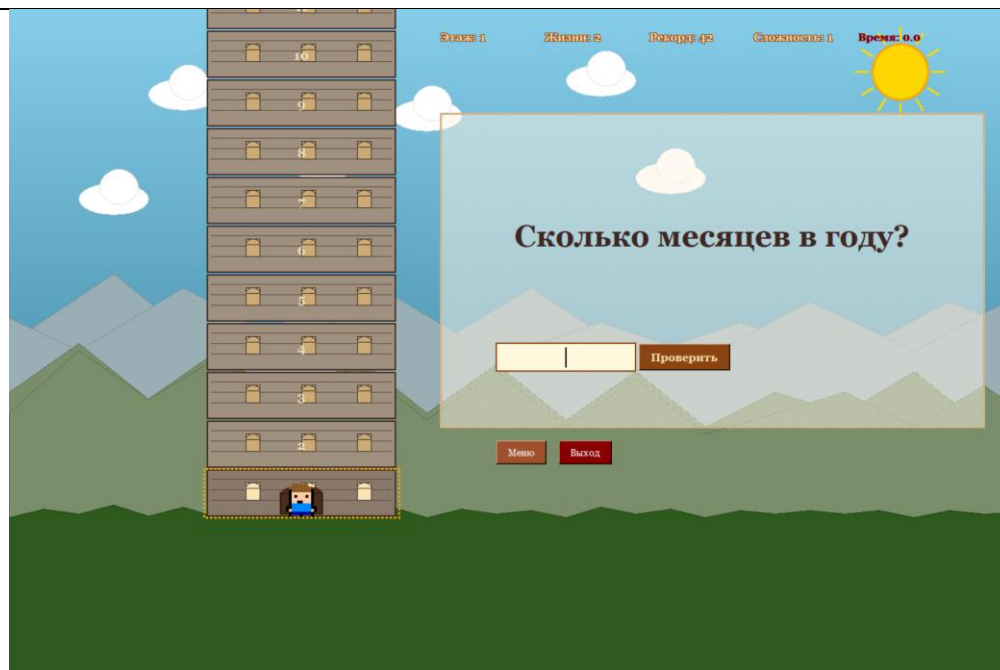


Рисунок 20 – Спуск персонажа на этаж ниже после неправильного ответа

11

Покрыта ветвь 11(T)

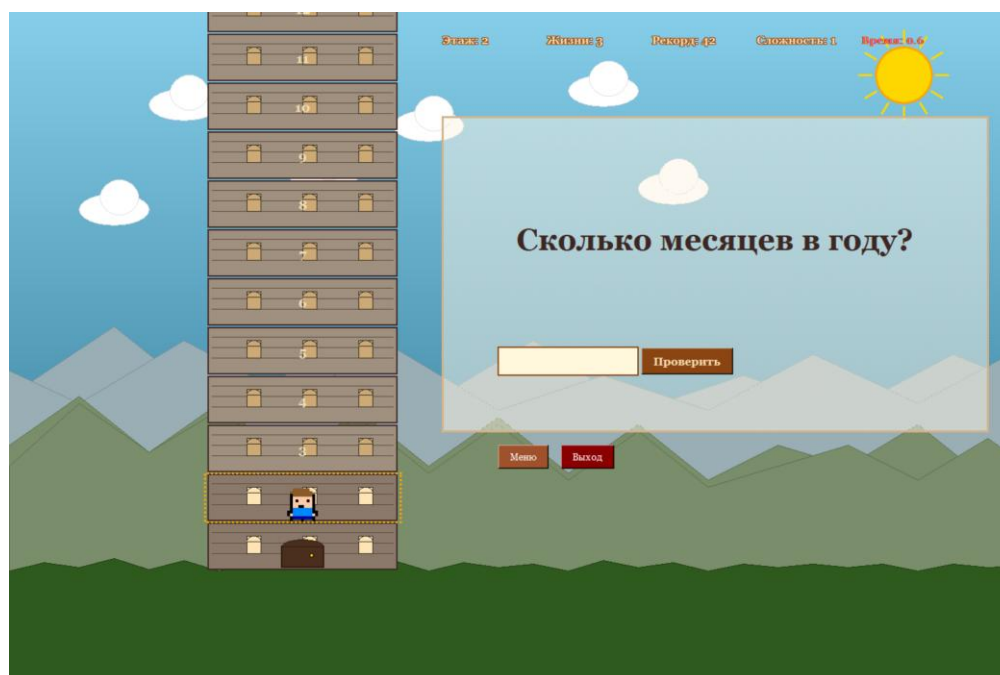


Рисунок 21 – Игровое окно с заканчивающимся таймером

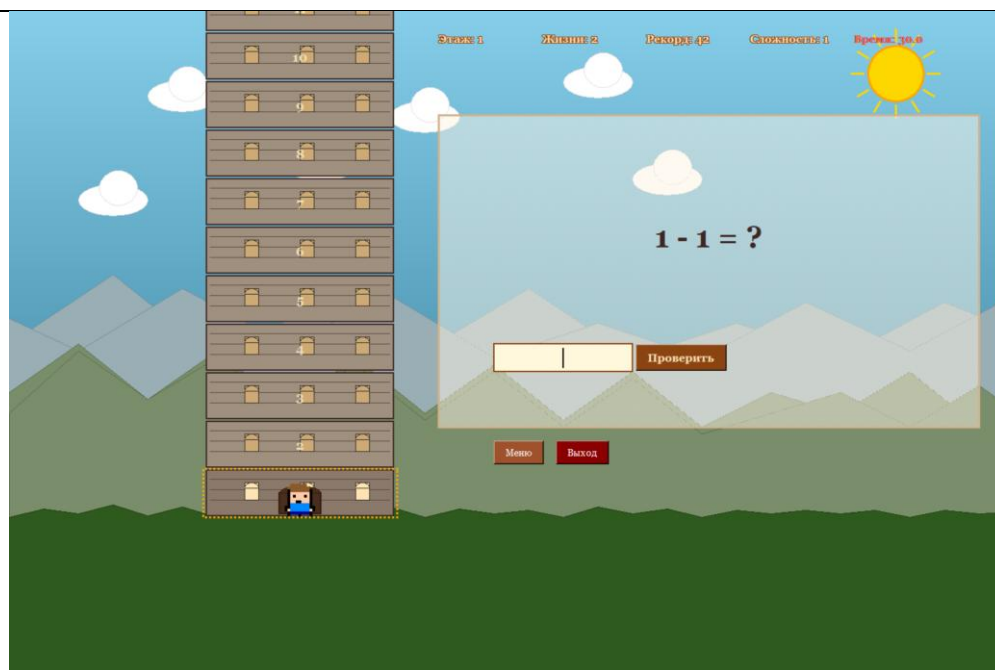


Рисунок 22 – Игровое окно после окончания времени

12

Покрыта ветвь 7(T)

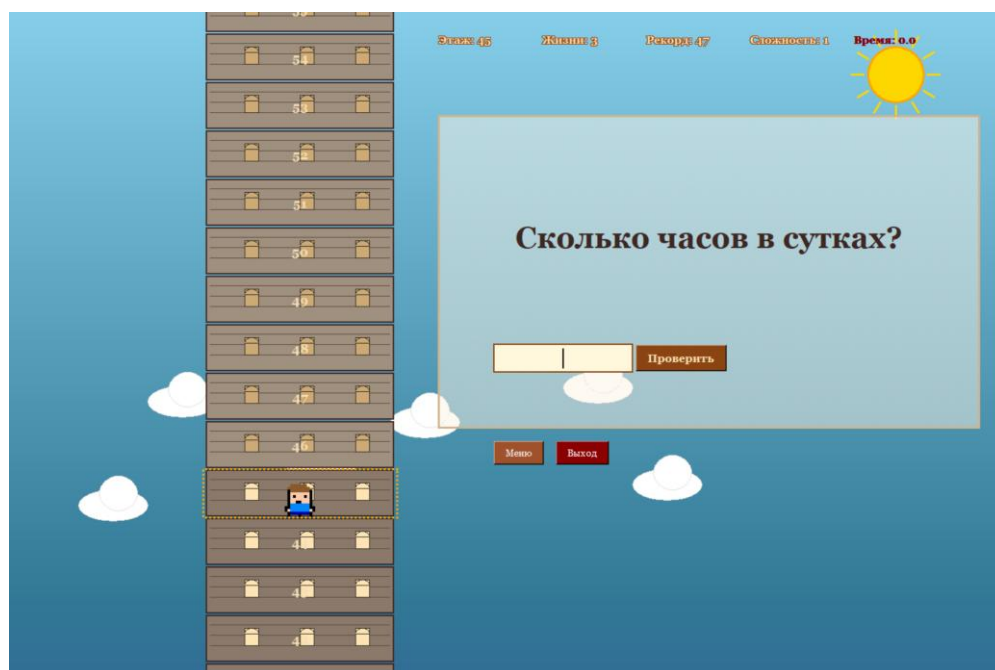


Рисунок 23 – Игровое окно с персонажем на 45 этаже



Рисунок 24 – Окно результатов с новым рекордом

13

Покрыта ветвь 12(T)

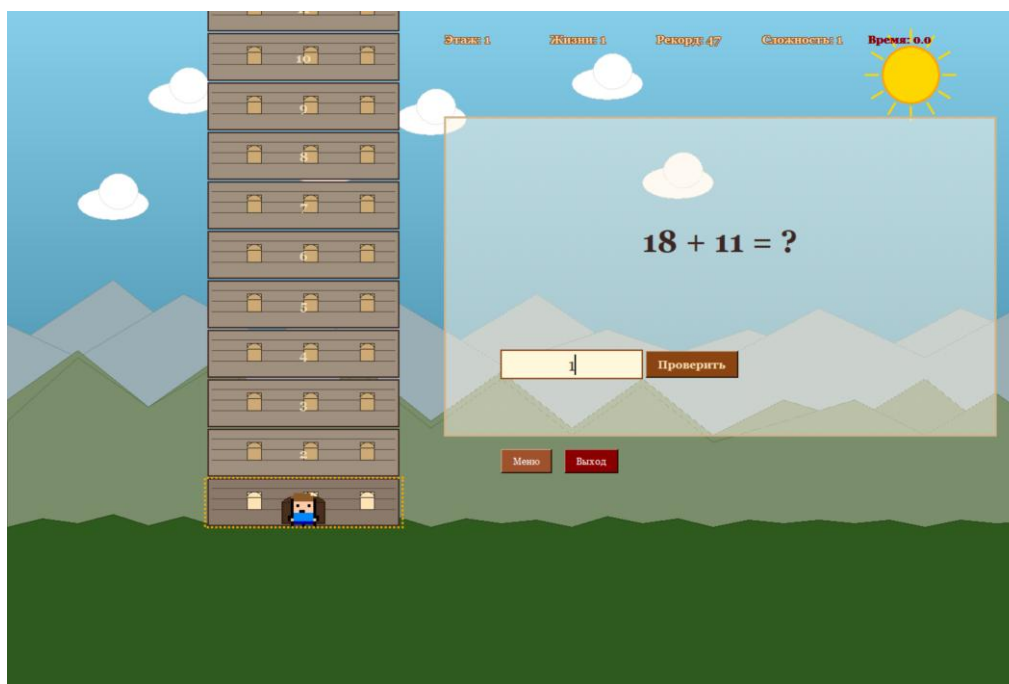


Рисунок 25 – Игровое окно с одной жизнью

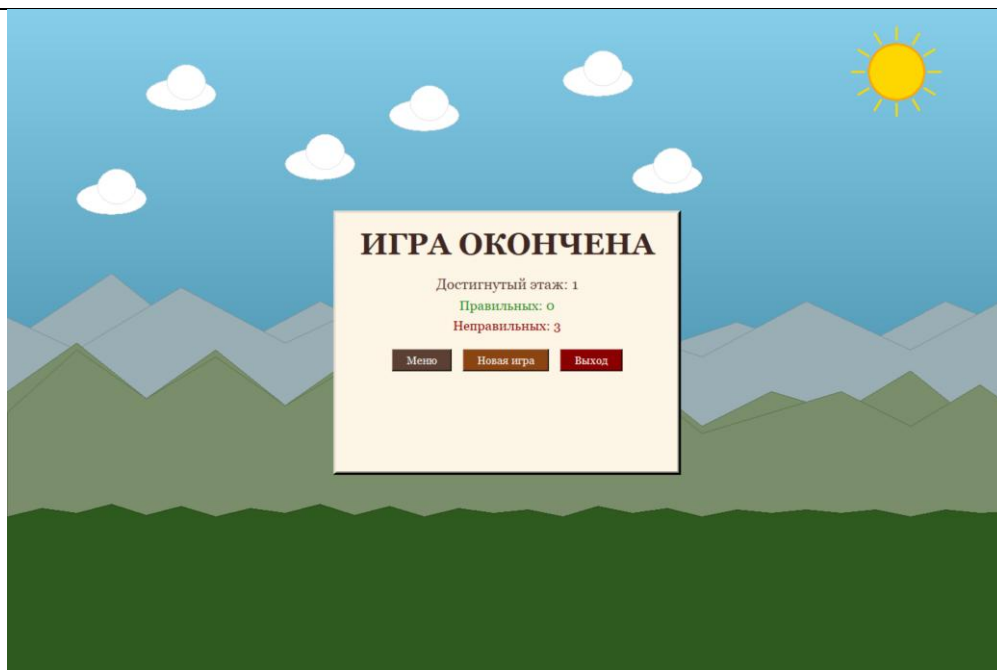


Рисунок 26 – Окно результатов после всех потраченных жизней

14

Покрыта ветвь 11(T)

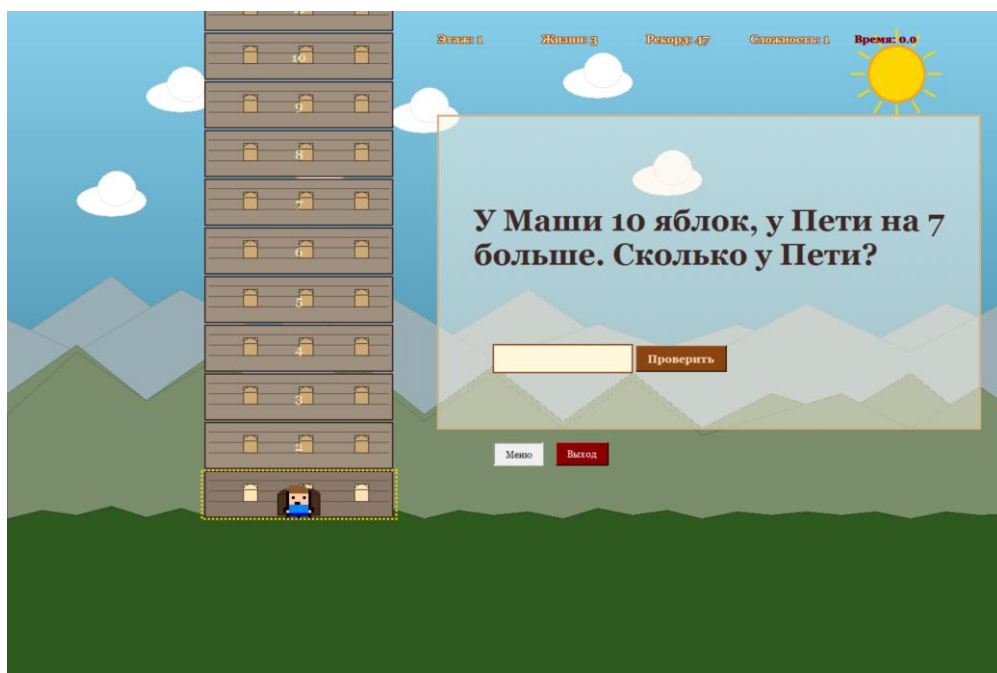


Рисунок 27 – Нажатие кнопки «Меню» в игровом окне

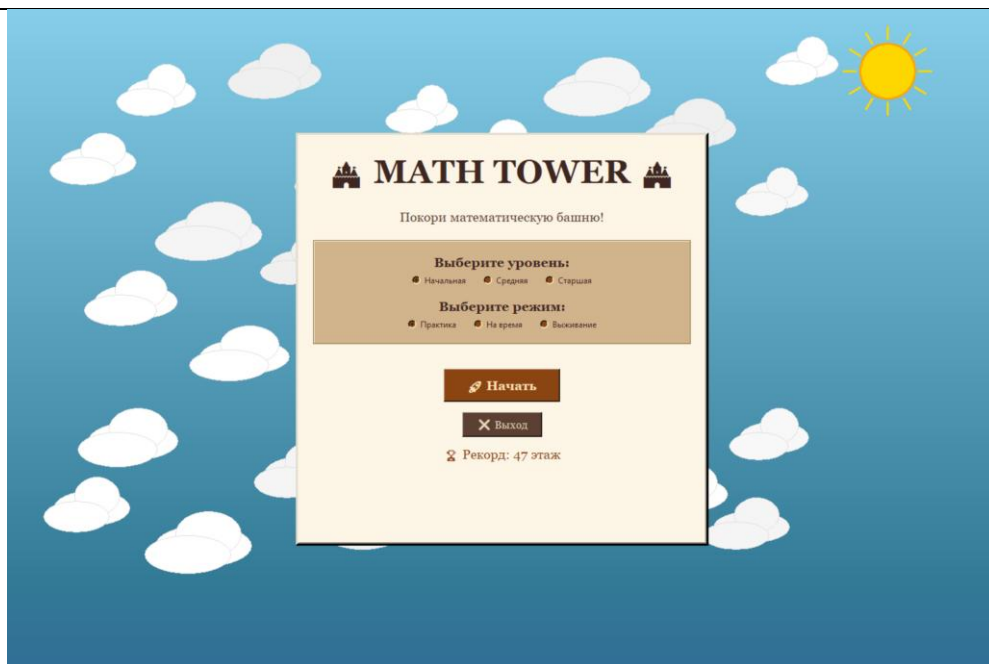


Рисунок 28 – Переход в стартовое окно после нажатия кнопки «Меню»

15

Покрыта ветвь 13(T)



Рисунок 29 – Нажатие кнопки «Новая игра» в окне результатов

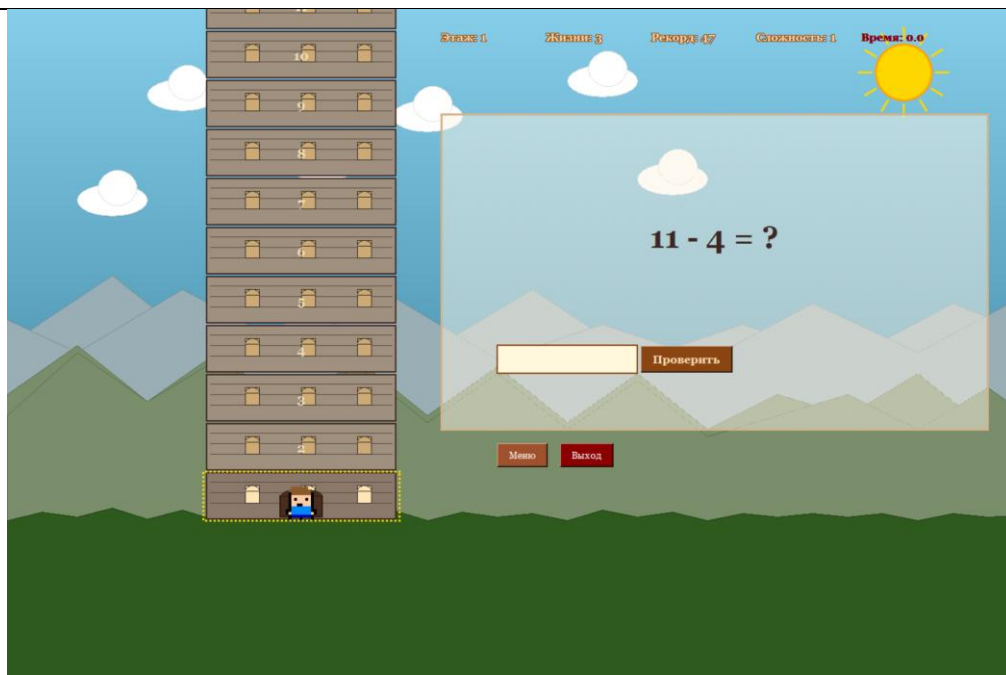


Рисунок 30 – Переход в игровое окно после нажатия кнопки «Новая игра»

16

Покрыта ветвь 14(T)

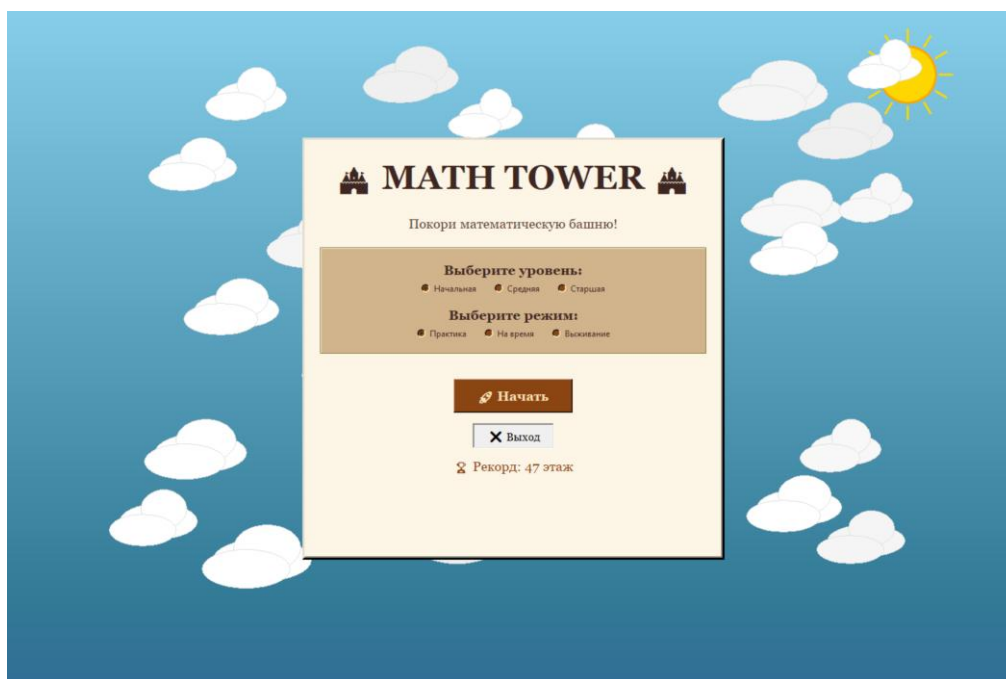


Рисунок 31 – Нажатие кнопки «Выход» в стартовом окне

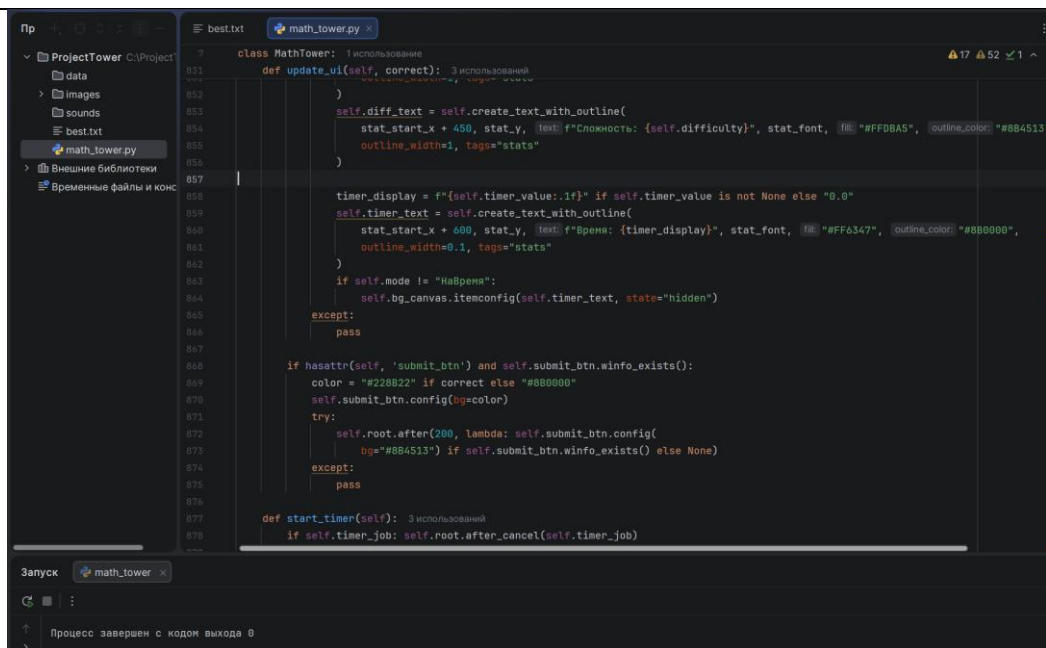


Рисунок 32 – Рабочий стол после выхода из игры

17

Покрыта ветвь 5(T)

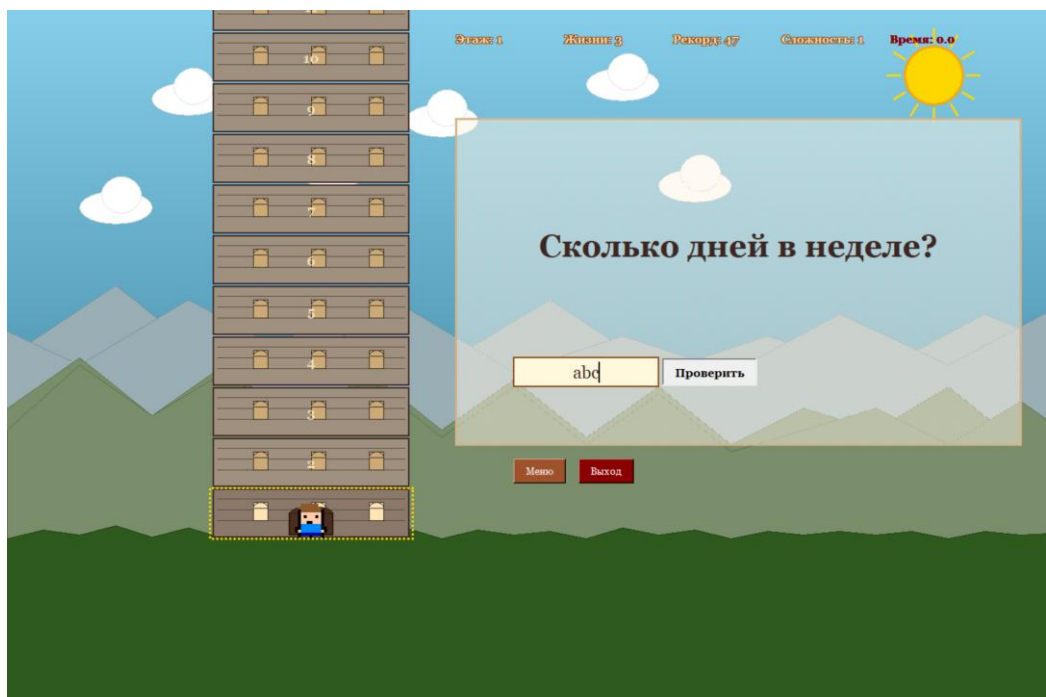


Рисунок 33 – Игровое окно с вводом некорректного ответа «abc»

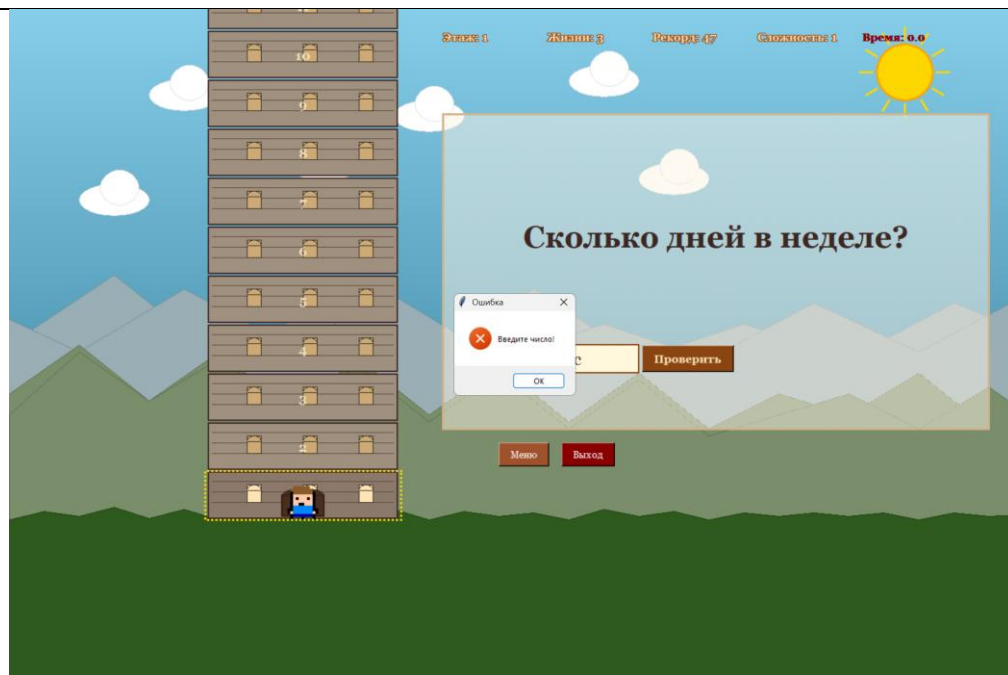


Рисунок 34 – Появление уведомления об ошибке

3 Заключение

В процессе прохождения практики выполнены следующие задачи:

- проведен сбор и анализ требований заказчика к программному средству;
- выполнена формализация выбранной предметной области;
- разработан проект программного средства;
- реализован проект программного средства;
- разработаны тесты и проведено тестирование программного средства;
- разработана и оформлена отчетная документация.

Таким образом, цели учебно-технологической (проектно-технологической) практики достигнуты.

В результате прохождения практики получены следующие профессиональные компетенции:

- способен разрабатывать языковые процессоры для формальных языков (ПК-7);
- способен создавать программные интерфейсы (ПК-8);
- способен использовать различные технологии разработки программного обеспечения (ПК-10).

4 Список использованных источников

1. Лутц М. Изучаем Python. Том I. [Электронный ресурс] – Режим доступа: <https://forum.rudtp.ru/resources/mark-lutts-izuchayem-python-tom-1-5-ye-izdaniye-2020-pdf.3272/> (дата обращения 14.06.2026).
2. Документация Python. Графический интерфейс пользователя с Tkinter. [Электронный ресурс] – Режим доступа: <https://pythonru.com/uroki/obuchenie-python-gui-uroki-po-tkinter> (дата обращения 14.06.2026).
3. Real Python. Python GUI Development With Tkinter. [Электронный ресурс] – Режим доступа: <https://realpython.com/python-gui-tkinter/> (дата обращения 14.06.2026).
4. GeeksforGeeks. Python Tkinter Tutorial. [Электронный ресурс] – Режим доступа: <https://www.geeksforgeeks.org/python-gui-tkinter/> (дата обращения 14.06.2026).
5. Stack Overflow. Tag: tkinter. [Электронный ресурс] – Режим доступа: <https://stackoverflow.com/questions/tagged/tkinter> (дата обращения 14.06.2026).